

ВСТУП

У сучасному світі стрімкий розвиток штучного інтелекту, зокрема великих мовних моделей (LLM), докорінно змінив підходи до обробки природної мови (Natural Language Processing, NLP). Якість роботи таких систем критично залежить від наявності великих, репрезентативних та якісно розмічених текстових даних, проте для української мови, що класифікується як мова з обмеженими ресурсами (low-resource language), проблема створення актуальних корпусів стоїть особливо гостро. Традиційні методи ручного формування корпусів не задовольняють потреби сучасних алгоритмів через низьку швидкість та високу вартість, тому на перший план виходять методи автоматичної агрегації даних із соціальних мереж та месенджерів – джерела «живої» мови, неологізмів та реакцій на актуальні події.

Актуальність теми дослідження полягає в необхідності удосконалення методів автоматичної побудови корпусів української мови, які б забезпечували не лише обсяг даних, але і їхню достовірність. Існуючі корпуси часто є статичними та застарілими, а автоматичний збір даних (web scraping) без належної верифікації призводить до накопичення «шуму», дублікатів та дезінформації. Особливо це стосується даних з месенджера Telegram, який, з одного боку, є цінним джерелом оперативної інформації, а з іншого – характеризується низьким рівнем фактологічного контролю.

Метою роботи є дослідження методів автоматичної обробки текстової інформації та розробка програмного забезпечення для створення актуального, верифікованого корпусу української мови на основі даних соціальних мереж (Telegram) та авторитетних новинних ресурсів.

Для досягнення цієї мети необхідно вирішити наступні завдання:

- провести аналіз існуючих методів автоматичної обробки текстової інформації (NLP), які застосовуються для побудови корпусів (токенізація, морфологічна лематизація, векторизація);

- дослідити алгоритми визначення семантичної подібності текстів – як статистичні (TF-IDF з косинусною мірою), так і нейромережеві (sentence embeddings на основі трансформерних моделей);
- розробити архітектуру системи, що включатиме модулі збору даних, лінгвістичної обробки, двох незалежних верифікаторів та ансамблевого комбінування;
- реалізувати програмні засоби для автоматичного збору неструктурованих даних з Telegram-каналів та еталонних даних з перевірених новинних RSS-стрічок;
- розробити та програмно реалізувати трикласову схему анотації (verified / uncertain / unverified), яка точніше відображає ступінь відповідності повідомлення з Telegram перевіреним новинам, ніж бінарна класифікація;
- реалізувати ансамблевий метод верифікації, що поєднує результати TF-IDF та семантичного верифікатора для підвищення стійкості рішення;
- провести експериментальне дослідження ефективності розроблених методів на власноруч розміченому наборі даних, обчислити метрики (Precision, Recall, F1, macro-F1, accuracy) та виконати пошук оптимальних гіперпараметрів за допомогою grid search.

Об'єктом дослідження є процеси автоматизованої обробки, аналізу та структурування великих масивів текстової інформації природною мовою.

Предметом дослідження виступають методи та алгоритми автоматичної побудови текстових корпусів, зокрема методи визначення семантичної подібності (Semantic Textual Similarity) для верифікації достовірності інформації з неструктурованих джерел.

Методи дослідження базуються на використанні мови програмування Python та її екосистеми для Data Science. Для збору даних застосовуються бібліотеки Telethon (Telegram API через MTProto), feedparser та BeautifulSoup (парсинг RSS-стрічок). Лінгвістична обробка реалізована засобами rymorphy3 для морфологічної лематизації. Для векторизації використано TF-IDF (scikit-learn) та щільні вектори моделі paraphrase-multilingual-MiniLM-L12-v2(sentence-transformers). Ефективність

оцінюється макроусередненою мірою F1, що враховує дисбаланс класів. Зберігання даних реалізовано у форматі JSONL.

Наукова новизна роботи полягає у розробці удосконаленого методу формування текстового корпусу, який, на відміну від існуючих підходів «сліпого» збору, інтегрує етап автоматичної фактологічної верифікації контенту з Telegram-каналів шляхом їх семантичного порівняння з еталонними джерелами. Це дозволяє автоматично маркувати ступінь достовірності даних у корпусі, підвищуючи його якість для задач машинного навчання.

Практичне значення отриманих результатів полягає у створенні програмного інструментарію, що дозволяє автоматизувати процес збору якісних мовних даних українською мовою. Сформований корпус та розроблене ПЗ можуть бути використані для покращення українських NLP-моделей, систем моніторингу інформаційного простору та виявлення дезінформації.

Зв'язок роботи з науковими програмами кафедри. Робота виконана відповідно до наукових напрямків кафедри програмної інженерії Харківського національного університету радіоелектроніки в галузі автоматичної обробки природної мови та виявлення дезінформації. Тематика дослідження пов'язана з науковими публікаціями керівника роботи доц. Н.А. Валенди, зокрема з роботою [21], присвяченою застосуванню великих мовних моделей для виявлення дезінформації в українських новинах.

Публікації. Основні результати кваліфікаційної роботи апробовано у тезах доповіді «Дослідження методів автоматичної обробки текстової інформації для створення корпусів української мови» на XXX Міжнародному молодіжному форумі «Радіоелектроніка і молодь в XXI столітті», секція 6.1 «Сучасні проблеми обчислювального та штучного інтелекту», м. Харків, 2026 р. За результатами доповіді отримано грамоту оргкомітету конференції.

1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ

1.1 Аналіз проблемної галузі дослідження

В епоху стрімкого розвитку штучного інтелекту та технологій обробки природної мови (Natural Language Processing, NLP) ключовим ресурсом, що визначає якість кінцевих моделей, стають дані. Ефективність сучасних великих мовних моделей (LLM) та алгоритмів машинного навчання безпосередньо залежить від наявності великих, репрезентативних та, що найважливіше, якісно розмічених текстових корпусів. Для української мови, яка в світовому контексті NLP все ще класифікується як мова з обмеженими ресурсами (low-resource language), проблема створення та наповнення актуальних корпусів залишається критичною [3]. Незважаючи на значний прогрес останніх років, кількість та якість доступних датасетів для української мови все ще значно поступається ресурсам для англійської чи інших європейських мов, що створює технологічний бар'єр для розвитку вітчизняних інтелектуальних систем.

Традиційні лінгвістичні корпуси, що створювалися вручну експертами-лінгвістами, протягом тривалого часу слугували еталоном якості. Проте в умовах експоненційного зростання обсягів інформації вони не можуть задовольнити потреби сучасних систем через обмежений обсяг та низьку швидкість оновлення. Існуючі анотовані набори даних для української мови часто є тематично обмеженими або специфічними, що не дозволяє ефективно використовувати їх для навчання моделей широкого профілю [2]. Це спонукає дослідників звертатися до методів автоматичного збору даних (web scraping) з відкритих джерел інтернету, що дозволяє швидко накопичувати великі масиви тексту.

Особливе місце серед джерел текстових даних займають соціальні мережі та месенджери, серед яких в українському сегменті інтернету домінує Telegram. Ця платформа є унікальним джерелом «живої» мови, що містить неологізми, сленг, специфічну військову лексику та миттєві реакції соціуму на актуальні події, які часто відсутні в академічних текстах чи художній літературі [7]. Такий матеріал є безцінним для навчання моделей, які мають розуміти сучасний контекст та розмовну мову.

Проте використання даних із соціальних мереж пов'язане з низкою серйозних викликів. По-перше, такі тексти характеризуються високим рівнем шуму: вони містять граматичні помилки, нестандартну пунктуацію, суржик та емодзі, що ускладнює їх автоматичну обробку. По-друге, в умовах інформаційної війни критичною проблемою стає поширення неперевіреної інформації, чуток та свідомої дезінформації [9]. Наповнення навчальних корпусів таким контентом без попередньої фільтрації створює ризик, відомий як "garbage in, garbage out": моделі, навчені на недостовірних даних, будуть генерувати галюцинації або поширювати фейки [12].

Таким чином, проблемна область дослідження охоплює не лише технічні аспекти автоматичного збору та очищення текстів, але й складну задачу забезпечення їхньої фактологічної якості. Створення корпусу «сліпим» методом автоматичного парсингу призводить до накопичення токсичного та маніпулятивного контенту [13]. Це актуалізує потребу в розробці нових методів, які б інтегрували процедури верифікації безпосередньо в процес створення корпусу.

1.2 Огляд існуючих підходів та обмежень

На сьогодні для української мови існує кілька значних корпусних проєктів, які використовуються як стандарти в галузі комп'ютерної лінгвістики. Найвідомішими та найбільш цитованими серед них є Генеральний регіонально-анотований корпус української мови (ГРАК) та корпус UberText.

Генеральний регіонально-анотований корпус української мови (ГРАК) є одним з найбільш репрезентативних академічних ресурсів. Він охоплює тексти з XIX по XXI століття, містить детальну метарозмітку та дозволяє досліджувати регіональні особливості мови. ГРАК є незамінним інструментом для лінгвістичних досліджень [2], проте його основним обмеженням у контексті навчання NLP-моделей є орієнтація на літературні, публіцистичні та наукові тексти, що проходять певний редакційний відбір. Процес наповнення та оновлення таких корпусів є

досить повільним, що не дозволяє відслідковувати динамічні мовні зміни в режимі реального часу.

UberText, у свою чергу, являє собою масштабний корпус, зібраний переважно з новинних стрічок та онлайн-видань. Завдяки своєму обсягу та сучасній лексиці він краще підходить для задач NLP [1]. Проте, як зазначають дослідники, навіть такі корпуси мають обмеження щодо покриття специфічних доменів та жанрів, а також можуть містити застарілу лексику, яка не повною мірою відображає сучасний стан мовного ужитку, що швидко змінюється під впливом соціально-політичних факторів.

Для наочного порівняння характеристик існуючих рішень та запропонованої системи, зведемо основні параметри до таблиці 1.1. Як видно з наведених даних, ключовою відмінністю розроблюваної системи є орієнтація на динамічний контент із соціальних мереж та наявність вбудованого модуля автоматичної верифікації достовірності.

Таблиця 1.1 – Порівняльна характеристика підходів до формування корпусів української мови

Характеристика	ГРАК (General Regionally Annotated Corpus)	UberText (Lang-uk)	Розроблювана система (Telegram-News Corpus)
Джерела даних	Художні, публіцистичні та наукові тексти (XIX–XXI ст.)	Новинні стрічки, онлайн-видання, сайти	Telegram-канали (соціальні мережі) та перевірені новинні сайти
Тип лексики	Літературна, нормативна, академічна	Офіційно-ділова, публіцистична	«Жива» розмовна мова, сленг, неологізми, військова лексика
Динамічність	Статичний (оновлюється рідко, вручну/напівавтоматично)	Статичний (фіксує стан мови на момент збору)	Динамічний (збір та оновлення у режимі реального часу)

Кінець таблиці 1.1

Характеристика	ГРАК (General Regionally Annotated Corpus)	UberText (Lang-uk)	Розроблювана система (Telegram-News Corpus)
Верифікація фактів	Забезпечується редакційним відбором джерел	Відсутня («сліпий» веб-скрапінг)	Автоматизована (Semantic Textual Similarity з еталонними джерелами)
Основний недолік	Повільне оновлення, відсутність оперативної лексики	Наявність «шуму» та застарілої інформації, відсутність перевірки на фейки	Необхідність складної попередньої обробки неструктурованого тексту

Процес автоматичного створення корпусів з веб-джерел зазвичай реалізується через стандартизований пайплайн. Він розпочинається з автоматичного обходу вебсторінок (web crawling) за заданими правилами або списком доменів. Наступним кроком є очищення контенту (boilerplate removal), яке передбачає відокремлення основного тексту статті від навігаційних елементів, реклами, HTML-тегів та службової інформації. Після цього застосовуються алгоритми дедуплікації для видалення повних або часткових копій текстів, а також фільтрація за мовою (language detection), щоб відсіяти іншомовний контент.

Головним недоліком існуючих підходів до формування корпусів із соціальних медіа є відсутність етапу фактологічної верифікації. При роботі з даними з платформ на кшталт Telegram, X чи Facebook, традиційні методи скрапінгу збирають увесь доступний контент, що відповідає мовним критеріям, не розрізняючи достовірні новини від суб'єктивних думок, чуток чи цілеспрямованих маніпуляцій [4]. Це призводить до того, що значна частина зібраного корпусу може складатися з інформаційного сміття.

Існуючі інструменти для автоматичного виявлення фейкових новин часто базуються на складних моделях глибокого навчання [8], що аналізують стиль тексту або патерни поширення інформації. Однак такі методи вимагають великих,

експертно розмічених навчальних вибірок (labeled datasets) із прикладами фейків та правдивих новин, яких для української мови критично бракує [10]. Це створює замкнене коло: для створення якісного детектору фейків потрібен розмічений корпус, а для створення чистого корпусу потрібен надійний детектор фейків.

Отже, існує нагальна науково-практична потреба в розробці альтернативних методів фільтрації, які б дозволяли автоматично верифікувати зібрані дані без необхідності у великих розмічених навчальних наборах. Перспективним напрямком є використання порівняння з довіреними джерелами (Reference-based Fact Checking), де достовірність тексту з соціальної мережі визначається через його семантичну відповідність матеріалам авторитетних новинних агентств.

1.3 Тенденції та перспективи розвитку

Сучасний розвиток технологій обробки природної мови демонструє чітку зміну парадигми: наукова спільнота поступово відходить від екстенсивного шляху, що базувався на гонитві за кількісними показниками обсягу даних, до інтенсивного, де пріоритетом стає їхня якість, чистота та надійності. Аналіз останніх досліджень у сфері комп'ютерної лінгвістики дозволяє стверджувати, що майбутнє корпусних технологій залежить від інтелектуальних систем, здатних не просто агрегувати текст, а й здійснювати його глибоку семантичну фільтрацію.

Однією з ключових тенденцій є інтеграція методів автоматичної перевірки фактів (Fact-Checking) безпосередньо у архітектуру систем збору даних. Якщо раніше верифікація розглядалася як окрема задача, що вирішується постфактум, то сьогодні дослідники пропонують вбудовувати модулі оцінки достовірності на ранніх етапах пайплайну. Найбільш перспективним інструментом для цього стає метод семантичної подібності текстів (Semantic Textual Similarity, STS). Він дозволяє порівнювати неструктуровані повідомлення із соціальних мереж з надійними джерелами інформації, визначаючи ступінь їхньої змістової тотожності, а не просто лексичного збігу [6]. Такий підхід дозволяє автоматизувати процес відділення фактів від фейків без залучення людських ресурсів. Зокрема, у роботі [21] показано перспективність використання великих мовних моделей для

виявлення дезінформації в українських новинах, що підтверджує загальну тенденцію переходу до інтелектуальних методів верифікації.

Паралельно відбувається активне впровадження гібридних підходів до аналізу тексту, що поєднують класичні статистичні методи з новітніми неймережевими архітектурами. Досвід показує, що використання перевірених часом алгоритмів, таких як TF-IDF, у тандемі з сучасними векторними представленнями слів (Word Embeddings) або трансформерними моделями (BERT, RoBERTa), дозволяє досягти оптимального балансу між швидкістю обробки та точністю результатів. Цей симбіоз є особливо ефективним при роботі з різнорідними даними, дозволяючи системі розуміти контекст повідомлень, написаних різними стилями, що є типовим для середовища Telegram [4].

Ще одним важливим вектором розвитку є перехід до концепції динамічних корпусів. Статичні набори даних, які фіксують стан мови на певний момент часу, швидко втрачають свою актуальність в умовах сучасного інформаційного потоку. Натомість виникає потреба у створенні "живих" корпусів, які оновлюються в режимі реального часу. Це стає критично важливим для моніторингу інформаційного простору під час кризових ситуацій, зокрема війни, коли швидкість появи нових термінів та загроз дезінформації вимагає миттєвої реакції.

Окремої уваги заслуговує зростаючий фокус світової науки на підтримці мов з обмеженими ресурсами (low-resource languages)[3], до яких належить і українська. Розробка методів крос-мовної верифікації [11] та адаптація багатомовних моделей відкриває нові можливості для українських дослідників. Це дозволяє використовувати глобальний досвід для вирішення локальних задач, зокрема створення спеціалізованих верифікованих корпусів, які стануть фундаментом для побудови безпечних та етичних національних систем штучного інтелекту [1].

Таким чином, перспективи розвитку предметної області пов'язані зі створенням автоматизованих систем, які виступатимуть не пасивними архіварами, а активними інтелектуальними фільтрами. Реалізація таких систем дозволить вирішити проблему інформаційного шуму та забезпечить дослідників якісними даними, необхідними для подальшого прогресу в галузі штучного інтелекту.

2 ОГЛЯД Й АНАЛІЗ ЛІТЕРАТУРНИХ, НАУКОВИХ ДЖЕРЕЛ

2.1 Огляд літературних джерел

У межах дослідження методів автоматичної побудови та верифікації корпусів української мови було проведено системний пошук та аналіз наукових публікацій. Метою огляду було виявлення сучасних підходів (State-of-the-Art) у сферах комп'ютерної лінгвістики, обробки природної мови (NLP) та автоматичної перевірки фактів (Automated Fact-Checking).

Для формування джерельної бази використовувалися провідні наукометричні платформи та цифрові бібліотеки: Google Scholar, ACL Anthology (Association for Computational Linguistics), IEEE Xplore, arXiv.org та ResearchGate. Пошук здійснювався за ключовими словами українською та англійською мовами: «NLP corpus creation», «low-resource languages», «Ukrainian language corpus», «automated fact-checking», «Telegram fake news detection», «semantic textual similarity».

Критерії відбору джерел включали:

- актуальність: пріоритет надавався роботам, опублікованим у період з 2019 по 2024 роки, оскільки галузь NLP розвивається надзвичайно стрімко (зокрема, з появою трансформерних моделей);
- релевантність: відбиралися дослідження, що стосуються специфіки української мови або методів роботи з неструктурованими даними соціальних мереж;
- наукова значущість: Враховувалися публікації у рецензованих журналах та матеріалах профільних конференцій (UNLP, LREC-COLING, EMNLP).

За результатами пошуку відібрані джерела були класифіковані за трьома тематичними групами.

Перша група - розробка та наповнення текстових корпусів. У цю групу увійшли фундаментальні роботи, що описують існуючі корпуси української мови та методологію їх створення. Особливу увагу приділено публікаціям Дмитра Чаплинського щодо корпусу UberText 2.0 (2023), який є стандартом для навчання сучасних мовних моделей [1], а також роботам Марії Шведової та співавторів щодо розвитку Генерального регіонально-анотованого корпусу української мови (ГРАК)

[2]. Також проаналізовано дослідження, що розглядають українську мову в контексті *low-resource languages* (мов з обмеженими ресурсами) та пропонують методи подолання дефіциту даних, наприклад, робота Haltiuk та ін. (2024) про шляхи перетворення української на мову з високими ресурсами [3].

Друга група - виявлення дезінформації та аналіз даних у Telegram. Ця група джерел висвітлює специфіку роботи з даними месенджерів. Важливими є матеріали воркшопів UNLP (Ukrainian Natural Language Processing Workshop), зокрема спільні задачі (Shared Tasks) 2024–2025 років, присвячені виявленню маніпуляцій та пропаганди в українському сегменті Telegram [4]. Роботи таких авторів, як Ustyianovych та Barbosa (2024), надають розуміння технічних підходів до класифікації текстів у Telegram-каналах та виявлення патернів поширення фейків.

Остання група - методи автоматичної перевірки фактів (Automated Fact-Checking). Третя група охоплює теоретичні та практичні роботи з автоматизації фактчекінгу. Сюди увійшли оглядові статті (surveys), наприклад, робота Guo et al. (2022) та Akhtar et al. (2023), які систематизують методи визначення правдивості на основі NLP [5]. Особливий інтерес становлять дослідження, що пропонують використання методів семантичної подібності (Semantic Textual Similarity) та трансформерних моделей (BERT, RoBERTa) для порівняння тверджень (claims) з доказами (evidence) [6].

Таким чином, сформована джерельна база охоплює як фундаментальні питання корпусної лінгвістики, так і прикладні аспекти боротьби з дезінформацією, що створює надійне підґрунтя для розробки власної системи верифікації.

2.2 Аналіз літератури

Аналіз відібраних джерел дозволяє глибше зрозуміти еволюцію методів обробки текстової інформації та виявити ключові тенденції у створенні мовних ресурсів для української мови. Дослідження у першій групі, присвяченій корпусній лінгвістиці, демонструють значний прогрес у подоланні дефіциту даних. Робота Дмитра Чаплинського, присвячена корпусу UberText 2.0 [1], є фундаментальною для розуміння сучасних підходів до масового збору даних (web crawling). Автор

детально описує методологію очищення текстів від «сміття» (boilerplate removal) та дедуплікації, що дозволило зібрати понад 3 мільярди токенів. Однак, аналіз показує, що основний акцент у цій роботі зроблено на лінгвістичній чистоті та обсязі, тоді як фактологічна достовірність контенту залишається поза межами автоматизованої перевірки. Це підтверджує тезу про те, що існуючі методики створення корпусів орієнтовані на форму, а не на зміст.

Паралельно, дослідження Марії Шведової та колег щодо корпусу ГРАК [2] ілюструють академічний підхід до анотації, де кожному тексту присвоюються детальні метадані, включаючи регіон походження та жанр. Хоча такий підхід забезпечує високу якість даних, він є трудомістким і складно масштабується для задач реального часу, таких як моніторинг новинних стрічок у месенджерах. Робота Haliuk та ін. [3] пропонує стратегії трансформації української мови з категорії low-resource у high-resource, наголошуючи на важливості використання автоматизованих пайплайнів та трансферного навчання (transfer learning) від багатших на ресурси мов.

Друга група джерел, що фокусується на аналізі Telegram, розкриває специфічні виклики роботи з цим середовищем. Матеріали воркшопів UNLP, зокрема результати спільних задач із виявлення маніпуляцій [4], свідчать про те, що тексти у Telegram мають унікальні характеристики: вони коротші, емоційно забарвлені та часто містять неявну пропаганду. Дослідники активно використовують трансформерні моделі (BERT, RoBERTa) для класифікації таких повідомлень. Проте більшість розглянутих робіт зосереджені на бінарній класифікації (фейк/правда) або виявленні мови ворожнечі на основі внутрішніх ознак тексту, без порівняння з зовнішніми джерелами. Це обмежує їхню ефективність у випадках, коли фейк написаний нейтральним стилем і виглядає як звичайна новина.

Третій напрямок досліджень – автоматична перевірка фактів – пропонує рішення, яке може компенсувати недоліки попередніх підходів. Оглядова робота Guo et al. [5] систематизує методи fact-checking і виділяє підхід на основі пошуку доказів (evidence retrieval) як найбільш перспективний. Суть методу полягає у

розбитті процесу на три етапи: виявлення твердження, пошук відповідних документів у базі знань та визначення вердикту. Дослідження FacTeR-Check [6] розвиває цю ідею, пропонуючи використовувати семантичну подібність (Semantic Textual Similarity) для зіставлення перевірюваних тверджень із заголовками авторитетних новин. Такий підхід дозволяє верифікувати інформацію не через аналіз стилю, а через перевірку наявності підтвердження у надійних джерелах.

Узагальнюючи аналіз літератури, можна стверджувати, що на сьогодні існують ефективні інструменти для збору даних (як в UberText) та потужні методи семантичного аналізу (як у Fact-Checking системах). Однак, інтеграція цих двох компонентів у єдину систему для автоматичного створення верифікованих корпусів української мови залишається недостатньо дослідженою. Більшість існуючих рішень вирішують ці задачі окремо, що відкриває простір для наукового пошуку в напрямку створення гібридних систем, які поєднують масовий збір даних із їхньою миттєвою семантичною верифікацією.

2.3 Оцінка актуальності та новизни

Проведений огляд та аналіз літературних джерел підтверджують беззаперечну актуальність дослідження в сфері створення верифікованих корпусів української мови. В умовах гібридної війни та інформаційного протистояння, український сегмент інтернету, і зокрема месенджер Telegram, перенасичений дезінформацією та маніпулятивним контентом. Існуючі підходи до створення корпусів, як-от UberText [1], хоча й вирішують проблему обсягу даних, не мають вбудованих механізмів перевірки правдивості. Це створює ризик використання «отруєних» даних (data poisoning) при навчанні національних моделей штучного інтелекту.

Аналіз літератури виявив суттєву прогалину: більшість методів детекції фейків [4] покладаються на класифікацію стилю тексту, що є недостатнім для виявлення якісно написаних фейків або рерайтів. Водночас, методи автоматичного фактчекінгу [5] розвиваються переважно для англійської мови і рідко застосовуються для задач масового збору даних у реальному часі.

Новизна нашого дослідження полягає у розробці гібридного підходу, який поєднує методи масового збору неструктурованих даних (web scraping) з методами автоматичної верифікації на основі семантичного порівняння з довіреними джерелами. На відміну від існуючих рішень, які аналізують текст ізольовано, ми пропонуємо використовувати метод Reference-based Verification, де кожне повідомлення з Telegram перевіряється на відповідність контенту авторитетних новинних ресурсів.

Інноваційним аспектом є застосування технік Semantic Textual Similarity (STS) та метрик косинусної подібності для автоматичного маркування ступеня довіри до кожного елемента корпусу. Це дозволяє перетворити процес створення корпусу з простого накопичення текстів на інтелектуальну селекцію фактів. Такий підхід не лише покращує якість самого корпусу, але й створює фундамент для розробки автономних систем моніторингу інформаційного простору, здатних працювати без постійного втручання людини-фактчекера.

Практична цінність роботи підсилюється орієнтацією на українську мову як low-resource language [3]. Запропонований метод дозволяє автоматично генерувати розмічені набори даних (пари «фейк-спростування» або «твердження-підтвердження»), які є критично необхідними для навчання наступних поколінь українських NLP-моделей.

2.4 Висновки з огляду

Проведена оцінка літературних джерел підтверджує, що обраний напрямок дослідження знаходиться на вістрі сучасної комп'ютерної лінгвістики. Аналіз показав, що хоча інструментарій для збору даних (scrapers) та інструментарій для аналізу тексту (NLP models) розвинені достатньо добре, їх ефективна комбінація для задачі верифікації корпусів української мови ще не реалізована повною мірою.

Ключові висновки з огляду літератури:

- дефіцит якісних даних: існуючі корпуси (ГРАК, UberText) є статичними і не покривають динамічний контент месенджерів;

- обмеженість методів: традиційні методи виявлення фейків вимагають великих розмічених вибірок, яких для української мови бракує;
- перспективність STS: методи семантичної подібності є найбільш адекватним інструментом для автоматичної верифікації без попереднього навчання на розмічених даних.

Таким чином, огляд літератури обґрунтовує доцільність розробки програмної системи, яка б автоматизувала процес збору даних з Telegram та їх верифікацію через порівняння з новинними сайтами. Це дозволить закрити існуючу наукову прогалину та створити інструмент, затребуваний як дослідниками даних, так і фахівцями з інформаційної безпеки.

3 ПОСТАНОВКА ЗАДАЧІ

Після аналізу предметної галузі та виявлення ключових проблем, пов'язаних зі створенням корпусів української мови, було визначено необхідність розробки нових підходів, які б поєднували автоматичний збір даних із їхньою фактологічною верифікацією. Аналіз літературних джерел показав, що існуючі рішення або фокусуються виключно на обсязі даних, ігноруючи їхню якість, або використовують складні моделі для детекції фейків, які потребують розмічених навчальних вибірок, відсутніх для української мови.

Метою роботи є розробка програмного забезпечення для автоматизованого створення верифікованого корпусу української мови, що базується на семантичному порівнянні неструктурованих повідомлень із Telegram-каналів з еталонними даними авторитетних новинних ресурсів.

Основна науково-технічна проблема полягає у неможливості використання стандартних методів веб-скрапінгу для наповнення якісних лінгвістичних корпусів через високий рівень інформаційного шуму та дезінформації в соціальних мережах. Поставлена задача передбачає створення системи, здатної в автоматичному режимі виконувати повний цикл обробки інформації: від збору "сирих" даних до їх очищення, лінгвістичної нормалізації та верифікації.

Для досягнення поставленої мети необхідно вирішити ряд технічних завдань:

- розробка модуля збору даних (Data Collection) – реалізація асинхронного механізму отримання повідомлень з Telegram-каналів у реальному часі засобами бібліотеки Telethon та збору новин з авторитетних джерел через RSS-стрічки для формування еталонної бази знань;
- розробка модуля попередньої обробки (Preprocessing), тобто створення алгоритмів очищення тексту від специфічного шуму месенджерів, Unicode-нормалізації (NFC) та морфологічної лематизації засобами rymorphy3 для приведення флексивних форм української мови до канонічного вигляду;

- реалізація двох незалежних алгоритмів верифікації. Перший це TF-IDF верифікатор із корпусно-рівневою IDF, бонусом за збіг іменованих сутностей та штрафом за низьке покриття ключовими словами. Другий це семантичний верифікатор на основі багатомовної моделі paraphrase-multilingual-MiniLM-L12-v2 із м'яким попереднім фільтром та бонусом за корроборацію декількома джерелами;
- реалізація ансамблевого методу, який лінійно комбінує оцінки TF-IDF та семантичного верифікатора з оптимальними ваговими коефіцієнтами;
- впровадження трикласової схеми анотації (verified / uncertain / unverified) із двома пороговими значеннями замість бінарного рішення;
- проєктування схеми зберігання даних у форматі JSONL.

Для формалізації функціональних вимог до розроблюваного програмного забезпечення та визначення меж системи було побудовано діаграму варіантів використання (Use Case Diagram). Вона відображає основні сценарії взаємодії користувача (дослідника) та автоматизованих процесів із компонентами системи. Взаємодію акторів та прецедентів наведено на рис. 3.1.

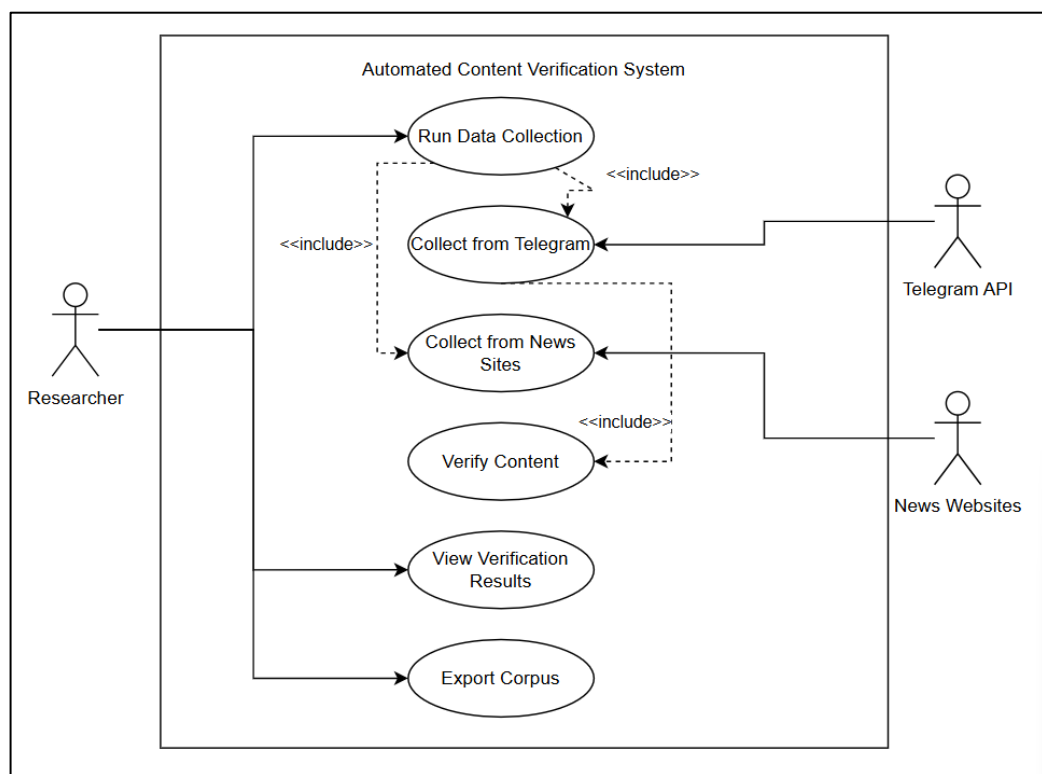


Рисунок 3.1 – Діаграма варіантів використання системи (рисунок створено самостійно)

Обґрунтування вибору інструментів реалізації базується на вимогах до швидкодії обробки текстів та гнучкості роботи з неструктурованими даними. В якості основної мови програмування обрано Python через наявність потужної екосистеми бібліотек для Data Science (scikit-learn, pandas) [14] та для роботи з нейромережевими моделями (sentence-transformers) [15]. Для взаємодії з Telegram API використано бібліотеку Telethon [17], яка реалізує протокол MTProto та підтримує асинхронну роботу, що критично важливо для обробки великих потоків повідомлень.

Для морфологічної лематизації української мови застосовано бібліотеку rymorphy3 разом зі словником rymorphy3-dicts-uk. Вибір саме цього інструменту, а не bag-of-words лематизаторів типу spaCy, обґрунтований кращим покриттям флексивних форм української мови: rymorphy3 використовує морфологічний словник, побудований на основі словника АОТ, який точніше визначає лему для відмінкових та родових форм (наприклад, «Зеленського», «Зеленському», «Зеленським» зводяться до однієї леми «зеленський»), що є критичним для україномовних текстів через їхню високу флексивність.

Для зберігання даних обрано формат JSONL (JSON Lines) – текстовий формат, де кожен рядок містить незалежний JSON-об'єкт. На відміну від СУБД MongoDB, що передбачалася в первісному плані, JSONL не потребує запуску окремого сервера бази даних, забезпечує природну відтворюваність експериментів (файл можна напряму увімкнути в репозиторій або передати), підтримує потокову обробку (читання рядок за рядком без завантаження всього датасету в пам'ять) і є фактичним стандартом обміну розміченими даними в спільноті NLP. Цей вибір спрощує дослідницький workflow без втрати функціональності, оскільки система не має навантаження тисяч одночасних користувачів, для якого справді потрібна повноцінна СУБД.

Після реалізації програмного забезпечення необхідно створити тестове середовище для проведення експериментального дослідження. Воно включатиме вибірку з кількох тисяч повідомлень із популярних українських новинних

Telegram-каналів та відповідний пул новин з перевірених сайтів (наприклад, «Українська правда», «Суспільне»).

Для оцінки ефективності розроблених методів використано стандартні метрики задач класифікації:

- Precision (точність) – частка повідомлень, правильно класифікованих системою у відповідний клас, серед усіх повідомлень, що отримали такий клас;
- Recall (повнота) – частка повідомлень певного класу, які система зуміла правильно ідентифікувати, серед усіх повідомлень цього класу в розміченій вибірці;
- F1-Score – гармонічне середнє Precision та Recall, що дозволяє оцінити загальний баланс алгоритму для кожного окремого класу;
- macro-F1 – незважене усереднення F1-Score за всіма трьома класами (verified, uncertain, unverified). Same macro-F1 обрано як основну оптимізаційну метрику, оскільки розмічена вибірка є істотно незбалансованою (клас «unverified» становить 56,7 %, клас «uncertain» – лише 9,2 %), і пряма оптимізація асигасу надмірно зміщувала б рішення на користь домінантного класу;
- асигасу – загальна частка правильно класифікованих зразків, що використовується як допоміжна метрика;
- матриця похибок (confusion matrix) – дозволяє якісно проаналізувати, які саме класи плутаються між собою, що особливо важливо для розуміння поведінки системи на межі прийняття рішень;
- час обробки одного повідомлення – для оцінки придатності до експлуатації в режимі, наближеному до реального часу.

Очікувані результати включають створення діючого програмного прототипу, який дозволить автоматично формувати корпус текстів із розміткою достовірності. Це вирішить проблему наповнення українських лінгвістичних баз актуальними даними та надасть дослідникам інструмент для швидкої фільтрації інформаційного простору.

4 ТЕОРЕТИЧНЕ ДОСЛІДЖЕННЯ

4.1 Математичні моделі представлення та порівняння текстів

Ключовим етапом роботи системи автоматичної верифікації новин є перетворення текстової інформації, яка за своєю природою є неструктурованою, у числовий вигляд, придатний для математичного аналізу. Цей процес називається векторизацією. Для вирішення задачі порівняння новин з Telegram та офіційних джерел було обрано модель "мішка слів" (Bag of Words) з використанням зважування TF-IDF (Term Frequency-Inverse Document Frequency) [19].

Вибір методу TF-IDF зумовлений кількома факторами:

- інтерпретованість: на відміну від "чорних скриньок" глибокого навчання (deep learning embeddings), результати TF-IDF легко пояснити: ми точно знаємо, які слова вплинули на схожість текстів;
- обчислювальна ефективність: метод не потребує потужних графічних процесорів (GPU) для навчання та інференсу, що дозволяє системі працювати в режимі реального часу навіть на звичайному серверному обладнанні;
- стійкість до шуму: при правильному налаштуванні (видалення рідкісних слів) TF-IDF ефективно фільтрує "сміттєві" слова, характерні для соціальних мереж.

Нехай $D = \{d_1, d_2, \dots, d_N\}$ – множина документів (новин) у нашому корпусі. Словник корпусу V складається з усіх унікальних слів (термінів), що зустрічаються в документах. Кожен документ d_j представляється як вектор у багатовимірному просторі розмірністю $|V|$, як показано у формулі (4.1):

$$\vec{d}_j = (w_{1,j}, w_{2,j}, \dots, w_{|V|,j}) \quad (4.1)$$

де $w_{i,j}$ – вага i -го терміна (t_i) в документі d_j .

Вага $w_{i,j}$ обчислюється як добуток двох компонентів за формулою (4.2):

$$w_{i,j} = TF(t_i, d_j) \times IDF(t_i, D) \quad (4.2)$$

Term Frequency (TF) визначає частоту терміна в конкретному документі. Вона показує, наскільки слово важливе для опису змісту саме цього тексту. Для уникнення зміщення в бік довгих текстів використовується нормована частота за формулою (4.3):

$$TF(t_i, d_j) = \frac{n_{i,j}}{\sum_k n_{k,j}} \quad (4.3)$$

де $n_{i,j}$ – кількість входжень терміна t_i в документ d_j ;

$\sum_k n_{k,j}$ – загальна кількість слів у документі.

Inverse Document Frequency (IDF) оцінює загальну важливість слова для всього корпусу. Вона зменшує вагу слів, які зустрічаються занадто часто (наприклад, загальновживані слова "рік", "сьогодні", "Україна") і не несуть унікальної інформації для розрізнення документів. IDF обчислюємо за формулою (4.4):

$$IDF(t_i, D) = \log \left(\frac{|D|}{|\{d \in D: t_i \in d\}|} \right) \quad (4.4)$$

де $|D|$ – загальна кількість документів у корпусі;

$|\{d \in D: t_i \in d\}|$ – кількість документів, що містять термін t_i .

Таким чином, найбільшу вагу отримують слова, які часто зустрічаються в конкретній новині, але рідко – в інших новинах корпусу. Це дозволяє виділити унікальні ключові слова (наприклад, прізвища, географічні назви, специфічні терміни), що описують подію.

Після перетворення текстів у вектори виникає задача визначення ступеня їхньої семантичної близькості. Для цього використовується косинусна міра подібності. На відміну від евклідової відстані, яка залежить від довжини векторів (а отже, від довжини текстів), косинусна міра оцінює кут між векторами [20]. Це критично важливо, оскільки новина в Telegram може бути коротким

повідомленням, а стаття на сайті – розгорнутим аналітичним матеріалом, але вони описують одну й ту ж подію.

Коефіцієнт подібності $Sim(\vec{A}, \vec{B})$ між вектором новини з Telegram ($\vec{A}$) та вектором новини з перевіреного джерела ($\vec{B}$) обчислюється за формулою (4.5):

$$Sim(\vec{A}, \vec{B}) = \cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \cdot \|\vec{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (4.5)$$

Результат набуває значень у діапазоні $[0,1]$ (оскільки елементи векторів TF-IDF невід’ємні):

- 1: вектори колінеарні, тексти ідентичні або складаються з однакових ключових слів;
- 0: вектори ортогональні, тексти не мають спільних значущих слів.

У розробленій системі цей показник використовується для прийняття рішення про верифікацію: якщо максимальне значення подібності між новиною з Telegram та будь-якою новиною з бази `trusted_news` перевищує встановлений поріг τ (threshold), новина вважається підтвердженою.

Окрім класичного статистичного підходу TF-IDF, у роботі також застосовано альтернативну модель представлення текстів – щільні семантичні вектори (sentence embeddings), отримані за допомогою попередньо натренованої трансформерної моделі. На відміну від розрідженого вектора TF-IDF розмірністю $|V|$ (де $|V|$ – потужність словника, що для україномовного корпусу новин зазвичай становить десятки тисяч термінів), семантичні моделі генерують щільний вектор фіксованої розмірності d (для обраної в роботі моделі – $d = 384$), у якому семантично близькі тексти розташовані ближче в евклідовому просторі.

Принципова відмінність двох підходів полягає в природі семантики, яку вони фіксують. TF-IDF чутливий до лексичного збігу: два тексти отримують високу подібність лише якщо вони використовують спільні (лематизовані) слова. Перефразування, синонімія або зміна порядку слів зменшують оцінку TF-IDF навіть за збереження змісту. Натомість sentence embeddings, отримані з

нейромережевої моделі, навченої на парах семантично еквівалентних речень, фіксують семантичну близькість: пари «Президент підписав закон про підвищення мінімальної зарплати» та «Глава держави затвердив новий розмір мінімальної оплати праці» отримують високу косинусну подібність у векторному просторі MiniLM, навіть не маючи жодного спільного значущого слова на лексичному рівні (після лематизації).

У роботі обрано модель `paraphrase-multilingual-MiniLM-L12-v2` з бібліотеки `sentence-transformers` [22] як компроміс між якістю та обчислювальною вартістю: модель є відносно компактною (≈ 400 МБ, 12 шарів трансформера, 384 вимірів виходу), підтримує 50+ мов включно з українською, і навчена на парах перефразувань, що робить її природно придатною для задачі визначення семантичної подібності (Semantic Textual Similarity, STS).

Косинусна подібність застосовується до векторів обох типів за єдиною формулою (4.5). Проте діапазон отриманих значень суттєво відрізняється: для TF-IDF косинусна подібність завжди невід'ємна та практично рідко перевищує 0,5–0,6 для змістовно споріднених текстів через розрідженість векторів; для MiniLM значення зазвичай перевищують 0,7 навіть для слабо споріднених речень, оскільки щільні вектори не мають нульових компонент. Цю відмінність необхідно враховувати при підборі порогів – для двох методів вони калібруються незалежно (див. підрозділи 4.3.4 та 4.3.5).

4.2 Алгоритмічні аспекти попередньої обробки даних (NLP Pipeline)

Оскільки вхідні дані з Telegram характеризуються високим рівнем «шуму» (емодзі, сленг, специфічна пунктуація, посилання, рекламні інтеграції), критично важливим етапом є попередня обробка тексту (Data Preprocessing). Без якісного очищення та нормалізації математичні моделі (такі як TF-IDF) не зможуть коректно сформувати вектори, що призведе до значного розрідження матриці ознак та зниження точності верифікації. Розроблений конвеєр обробки даних (NLP pipeline) реалізовано мовою Python з використанням бібліотек `re` та `rumorphy3` і складається з кількох послідовних етапів.

Першим кроком є нормалізація тексту (Text Normalization). Цей етап передбачає зведення тексту до канонічного вигляду, що є фундаментом для подальшої обробки. Зокрема, виконується Unicode Normalization (форма NFC), що гарантує ідентичність байтових кодів для візуально однакових символів, усуваючи проблему дублювання термінів через різне кодування. Також застосовується Case Folding – приведення всіх символів до нижнього регістру, що дозволяє алгоритму ідентифікувати слова, написані в різному регістрі, як один і той самий токен.

Наступним етапом є очищення від «шуму» (Noise Removal). Специфіка Telegram-каналів полягає у великій кількості службових елементів, які не несуть семантичного навантаження для задачі фактчекінгу. За допомогою регулярних виразів (Regular Expressions) з тексту видаляються URL-посилання, які часто є рекламними, теги користувачів (@username), службові команди, емодзі та спецсимволи. Також усуваються залишки Markdown-розмітки, що використовуються для форматування в месенджері. Це дозволяє очистити текст від елементів, які можуть заважати роботі статистичних моделей.

Далі виконується токенізація (Tokenization) – процес розбиття тексту на окремі атомарні одиниці (токени). Для української мови це є нетривіальною задачею через наявність апострофів і складених слів через дефіс. Використання стандартних розділювачів може призвести до помилкового розбиття слів. У системі застосовується розбиття за пробілами після попереднього очищення з регулярними виразами, що враховують особливості української орфографії: апостроф зберігається всередині слова, дефіс – у складених словах, а знаки пунктуації відокремлюються в окремі токени та потім відкидаються на етапі фільтрації.

Критично важливим етапом для зменшення розмірності векторного простору є морфологічна лематизація (Morphological Lemmatization). Це процес зведення слова до його початкової, словникової форми (леми) на основі морфологічного аналізу. Оскільки українська мова належить до високофлексивних, словформи інтенсивно змінюються за відмінками, родами, числами, часами та особами. Без лематизації різні форми одного слова (наприклад, «президент», «президента», «президентом», «президентами») створювали б окремі виміри у векторі TF-IDF,

спотворюючи статистичну частотність та підвищуючи розрідженість матриці ознак.

У системі для лематизації застосовано бібліотеку `rumorphy3` [16] разом зі словником `rumorphy3-dicts-uk`. Цей інструмент будується на принципах словникової морфології (на відміну від статистичних та нейромережових лематизаторів) і використовує адаптацію словника АОТ для української мови. Для кожного токена `rumorphy3` повертає набір можливих лем разом із морфологічними тегами; у процесі обробки обирається найбільш ймовірна нормальна форма (`normal_form`). Такий підхід коректно обробляє ключові випадки україномовних текстів: відмінкові форми власних назв («Зеленського», «Зеленському» → «зеленський»), узгоджені форми прикметників («українські», «українських» → «український»), часові форми дієслів («атакували», «атакують», «атакував» → «атакувати»). Зведення словоформ до єдиної леми суттєво підвищує якість роботи метрики косинусної подібності, оскільки дозволяє правильно зіставляти морфологічно споріднені слова в повідомленні з Telegram та статті-першоджерелі.

Альтернативні підходи на основі бібліотеки `sraCy` (модель `uk_core_news_sm`) розглядалися на етапі проєктування, проте від них було відмовлено. Українські моделі `sraCy` на момент дослідження мають нижчу якість лематизації для високофлексивних форм, оскільки використовують переважно статистичну апроксимацію, тоді як `rumorphy3` спирається на повний морфологічний словник, що дає вищу точність на іменниках, прикметниках та власних назвах – категоріях, які найчастіше визначають інформативність новинного тексту.

Завершальним етапом є фільтрація стоп-слів (Stop-word Removal). Цей процес передбачає видалення слів, що мають високу частоту в мові, але низьку інформативну цінність. До них належать загальномовні стоп-слова (прийменники, сполучники, частки) та доменно-специфічні слова, характерні для новинних Telegram-каналів (наприклад, «*підпишись*», «*посилання*», «*канал*»). Їх видалення дозволяє сфокусувати увагу алгоритму на суті новини та зменшити розмір словника корпусу на 30-40% без втрати змістовної інформації, що позитивно впливає на швидкодію та точність системи верифікації.

4.3 Розробка методу автоматичної верифікації

Основним функціональним елементом системи є модуль верифікації, який класифікує новини з Telegram на «підтверджені» та «непідтверджені». Цей процес базується на гіпотезі, що правдива резонансна подія, описана в соціальних мережах, з високою ймовірністю буде висвітлена авторитетними новинними агентствами протягом короткого проміжку часу.

Метод верифікації реалізовано як послідовний алгоритм, що складається з декількох кроків.

4.3.1 Формування пошукового простору (time window filtering)

Пряме порівняння кожного повідомлення з Telegram з усіма новинами в базі даних `trusted_news` є обчислювально неефективним, оскільки база постійно зростає. Для оптимізації швидкодії застосовується фільтрація за часовим вікном.

Нехай T_{tg} – час публікації повідомлення в Telegram. Для перевірки відбирається підмножина новин-кандидатів $D_{candidates}$ з довірених джерел, час публікації яких T_{news} потрапляє в інтервал, згідно з формулою (4.6):

$$D_{candidates} = \{d \in D_{trusted} \mid T_{tg} - \Delta t \leq T_d \leq T_{tg} + \Delta t\} \quad (4.6)$$

де Δt – параметр часового вікна (наприклад, 24 години).

Це дозволяє врахувати ситуації, коли новина в Telegram з'являється раніше за офіційну публікацію (оперативність месенджерів) або з невеликим запізненням.

4.3.2 Визначення найкращого відповідника (best match retrieval)

Для вектора новини з Telegram $\vec{V}_{tg}$ та множини векторів-кандидатів $\{\vec{V}_1, \vec{V}_2, \dots, \vec{V}_k\}$ з $D_{candidates}$ обчислюється косинусна подібність. Метою є знаходження новини-еталона, яка максимізує функцію подібності за формулою (4.7):

$$Score_{max} = \max_{d_i \in D_{candidates}} (\text{Sim}(\vec{V}_{tg}, \vec{V}_{d_i})) \quad (4.7)$$

4.3.3 Прийняття рішення (3-Class Thresholding)

Фінальний вердикт щодо статусу повідомлення виноситься на основі порівняння максимальної оцінки схожості $Score_{max}$ із двома пороговими значеннями: $\tau_{verified}$ та $\tau_{uncertain}$, причому $\tau_{verified} > \tau_{uncertain}$. Двопорогова схема відображає природну градацію відповідності між повідомленням з Telegram та новинами з авторитетних джерел: повна відповідність, часткова відповідність та відсутність відповідності.

Функція верифікації $Verify(d_{tg})$ визначається згідно з формулою (4.8):

$$Verify(d_{tg}) = \begin{cases} \text{verified,} & \text{якщо } Score_{max} \geq \tau_{verified}, \\ \text{uncertain,} & \text{якщо } \tau_{uncertain} \leq Score_{max} < \tau_{verified}, \\ \text{unverified,} & \text{якщо } Score_{max} < \tau_{uncertain}. \end{cases} \quad (4.8)$$

Семантика класів:

- Verified, новина-першоджерело повністю покриває повідомлення з Telegram (подія підтверджена);
- Uncertain, новина-першоджерело лише частково покриває повідомлення (подія мала місце, але деталі різняться, або висвітлення неповне);
- Unverified, у часовому вікні не знайдено семантично схожих публікацій з авторитетних джерел.

Важливо підкреслити, що клас uncertain не є оцінкою впевненості системи, а самостійною змістовою категорією: він описує реальну ситуацію часткового покриття, яку було б помилковим відносити як до verified, так і до unverified. Емпірично оптимальні значення порогів отримуються шляхом пошуку гіперпараметрів (grid search) на розміченій вибірці; для TF-IDF верифікатора найкращим виявилось поєднання $\tau_{verified} = 0,55$ та $\tau_{uncertain} = 0,50$.

Якщо новина отримує статус verified або uncertain, система зберігає посилання на знайдене першоджерело, що дозволяє користувачеві побачити підтвердження

інформації. Статус *unverified* означає, що в авторитетних джерелах не знайдено семантично схожих публікацій у заданому часовому проміжку, що може свідчити про дезінформацію, суб'єктивну думку автора каналу, надто локальну подію або фундаментальне обмеження методу для дуже коротких повідомлень.

4.3.4 TF-IDF верифікатор з удосконаленнями

Базова схема, описана в підрозділах 4.3.1–4.3.3, дає лише загальний контур алгоритму верифікації. Її пряма реалізація з застосуванням *TfidfVectorizer* бібліотеки *scikit-learn* у режимі «один екземпляр на пару порівнюваних текстів» виявляється істотно неефективною для задачі верифікації новин. У цьому підрозділі описано три ключових удосконалення TF-IDF верифікатора, що значно підвищують його якість.

Найважливішим вдосконаленням є розширення сфери розрахунку IDF з пари (повідомлення з Telegram, окрема стаття-кандидат) на всю множину кандидатів одночасно. У базовому варіанті, де векторизатор навчається лише на двох документах, формула IDF стає виродженою: будь-яке слово, що зустрічається в обох документах, отримує $IDF(t) = \log \frac{2}{2} = 0$; будь-яке слово, що зустрічається в одному з двох – $IDF(t) = \log \frac{2}{1} \approx 0,693$.

Тобто всі терміни мають практично однакову вагу, і TF-IDF фактично вироджується у звичайну *bag-of-words* модель. Це призводить до некоректної оцінки подібності: ультра-частотні слова на кшталт «Україна», «сьогодні», «повідомляє» отримують таку ж вагу, як рідкісні дискримінативні терміни – прізвища посадовців, назви населених пунктів, специфічні військові терміни.

Запропоноване вдосконалення полягає в тому, що векторизатор навчається на об'єднаному корпусі, що складається з лематизованого тексту повідомлення з Telegram та лематизованих текстів усіх відібраних на етапі часової фільтрації статей-кандидатів (зокрема, окремо для заголовка, тіла статті та конкатенації «заголовок + тіло»). За такої побудови IDF коректно відображає інформативність терміну в контексті актуальної новинної панорами: загальноновживані слова

отримують малу вагу, а дискримінативні – велику. Експериментально (див. розділ 5) це удосконалення має вирішальний вплив на якість верифікації.

Кожна стаття-кандидат подана трьома текстовими полями: заголовок, основний текст і їх конкатенація. Заголовок зазвичай містить найбільш концентровану інформацію про подію, проте через стислість може не мати спільних термінів з повідомленням з Telegram. Основний текст, навпаки, дає ширший контекст, але через свою довжину може містити багато інформаційного шуму, що знижує косинусну подібність. Тому для кожної пари (повідомлення, стаття) обчислюється три значення подібності, з яких береться максимум за формулою (4.9):

$$base_score = \max(\text{sim}(post, title), \text{sim}(post, body), \text{sim}(post, title + body)). \quad (4.9)$$

Така стратегія робить алгоритм стійким як до коротких заголовків, так і до довгих статей.

Якщо повідомлення з Telegram та стаття-кандидат містять спільні власні назви (прізвища, географічні назви, аббревіатури організацій), це є сильним сигналом семантичної близькості, який недостатньо враховується чистою TF-IDF подібністю. Для посилення такого сигналу базова оцінка домножується на коефіцієнт, що залежить від кількості спільних сутностей, як у формулі (4.10):

$$score_with_bonus = base_score \cdot (1 + \alpha \cdot |E_{post} \cap E_{article}|), \quad (4.10)$$

де E_{post} – множина іменованих сутностей у повідомленні;

$E_{article}$ – множина сутностей у статті;

α – коефіцієнт бонусу.

Експериментально (див. підрозділ 5.3) визначено оптимальне значення $\alpha = 0,14$ за критерієм macro-F1. Сутності виділяються спрощеним правилом: слова, що починаються з великої літери в оригінальному (нелематизованому) тексті, та

повністю великі аббревіатури завдовжки 2–6 символів (ЗСУ, НАТО, СБУ, США тощо). Після виділення кандидати лематизуються, що зводить різні відмінкові форми до єдиного представлення.

Якщо повідомлення з Telegram та стаття-кандидат мають дуже мало спільних значущих лем (менше встановленого порогу $MIN_KEYWORD_OVERLAP = 2$), оцінка подібності штрафується множенням на коефіцієнт 0,65. Цей механізм є додатковим запобіжником від хибнопозитивних спрацювань: навіть якщо TF-IDF показує помірну подібність на рівні 0,5–0,6 через випадковий збіг частотних термінів, низьке покриття змістовою лексикою свідчить про відсутність реального тематичного зв'язку.

Підсумкова оцінка для пари (повідомлення з Telegram, стаття-кандидат) (4.11):

$$final_score = base_score \cdot (1 + \alpha \cdot |E_{post} \cap E_{article}|) \cdot penalty_keyword, \quad (4.11)$$

$$\text{де } penalty_keyword = \begin{cases} 0,65, & \text{якщо } |K_{post} \cap K_{article}| < 2, \\ 1,00, & \text{інакше.} \end{cases}$$

Тут K_{post} та $K_{article}$ – множини значущих ключових лем повідомлення та статті відповідно (отримані після фільтрації стоп-слів та лем коротших за 4 символи).

Серед усіх кандидатів вибирається той, що максимізує $final_score$, після чого до нього застосовується трикласове правило прийняття рішення з підрозділу 4.3.3 з порогоми $\tau_{verified} = 0,55$ та $\tau_{uncertain} = 0,50$ (емпірично оптимальні значення за результатами grid search).

4.3.5 Семантичний верифікатор на основі MiniLM

Семантичний верифікатор реалізує альтернативну схему оцінювання подібності повідомлень з Telegram та новинних статей, яка спирається на щільні векторні представлення, отримані з трансформерної моделі paraphrase-multilingual-MiniLM-L12-v2. На відміну від TF-IDF верифікатора, що чутливий до лексичного збігу, семантичний підхід здатен виявляти змістовну спорідненість текстів навіть

за відсутності спільної лексики, що особливо актуально для перефразувань та різностилевих формулювань тієї ж події.

Як і в TF-IDF верифікаторі, на першому етапі застосовується фільтрація за часовим вікном (підрозділ 4.3.1) для відбору статей-кандидатів, опублікованих у часовому інтервалі ± 48 годин від часу публікації повідомлення з Telegram. Потім, на відміну від TF-IDF, де всі кандидати векторизуються одночасно для побудови коректної IDF, у семантичному верифікаторі застосовується *м'який попередній фільтр* на основі лексичного перекриття: усі кандидати в часовому вікні впорядковуються за спадною кількістю спільних значущих лем із повідомленням, після чого для подальшого аналізу обирається топ- K ($K = 5$ за замовчуванням).

Принципова різниця з TF-IDF верифікатором у тому, що м'який фільтр *не відкидає* кандидатів за критерієм мінімальної кількості спільних слів, а лише пріоритезує найбільш релевантних для подальшого, обчислювально витратного етапу нейромережевої векторизації. Це обґрунтовано двома факторами: по-перше, MiniLM здатна знаходити семантичну близькість між текстами без лексичного перекриття (саме в цьому її перевага), а пряме відкидання знеосмислило б цю властивість; по-друге, кодування трансформером є ресурсомістким, тому обмеження множини обчислюваних кандидатів є необхідним для забезпечення прийнятної швидкодії.

Для повідомлення з Telegram та кожного з K відібраних кандидатів обчислюється щільний 384-вимірний вектор (sentence embedding). Тексти подаються в модель у необробленому вигляді (без лематизації), оскільки трансформер навчений працювати з оригінальними словоформами і має внутрішню токенизацію WordPiece, що коректно обробляє флексивні форми. Косинусна подібність між векторами повідомлення та кожного кандидата обчислюється за стандартною формулою (4.5).

Аналогічно TF-IDF верифікатору, оцінка домножується на коефіцієнт, що залежить від кількості спільних іменованих сутностей. Проте у семантичному верифікаторі коефіцієнт менший – $\alpha_{semantic} = 0,04$ (порівняно з $\alpha_{tfidf} = 0,14$). Це обґрунтовано тим, що інформація про іменовані сутності частково вже закодована

у щільних векторах MiniLM, тому пряма надбавка має менше доданої цінності та при більшому коефіцієнті призводить до перевпевненості системи.

Семантичний верифікатор унікальний наявністю додаткового механізму – бонусу за корроборацію (multi-article corroboration bonus). Якщо в множині топ- K кандидатів існують декілька статей зі значною (хоч і не максимальною) подібністю до повідомлення з Telegram, це є додатковим сигналом достовірності події: одна й та сама подія, що висвітлюється кількома незалежними джерелами, заслуговує на вищий ступінь довіри.

Формально, нехай s^* – найвища оцінка серед K кандидатів (після застосування бонусу за сутності), а N_{corr} – кількість кандидатів, чия оцінка перевищує $\tau_{uncertain} \cdot 0,7$ (тобто є помірно подібними до повідомлення, але не є найкращим відповідником). Тоді отримуємо формулу (4.12):

$$score_corroborated = s^* + \beta \cdot N_{corr}, \quad (4.12)$$

де $\beta = 0,07$ – оптимальне значення коефіцієнту корроборації за результатами grid search.

Цей бонус є м'яким механізмом кумуляції доказів: одна стаття дає звичайну оцінку, дві-три статті – невеликий приріст, надмірно велика кількість слабких збігів – невелику, але обмежену надбавку.

Цікаво, що для TF-IDF верифікатора аналогічний експеримент показав оптимальне значення $\beta = 0,0$ (тобто бонус шкодить якості). Пояснення полягає у статистичній природі TF-IDF: у новинних потоках одночасно з'являється багато статей про близькі, але не пов'язані події (наприклад, регулярні щоденні зведення про обстріли різних областей). Чисто лексична подібність між ними є завищеною через спільну специфічну військову лексику, тому корроборація створює хибнопозитивні спрацювання. У щільному ж семантичному просторі такі регулярні новини частково розрізняються, і корроборація приносить корисний сигнал.

Як і у TF-IDF верифікаторі, остаточне рішення приймається порівнянням $score_corroborated$ з двома пороговими значеннями. Однак через істотно вищі

абсолютні значення косинусної подібності у щільному просторі MiniLM порогови для семантичного верифікатора калібруються окремо: за результатами grid search оптимальними значеннями виявилися $\tau_{verified} = 0,95$ та $\tau_{uncertain} = 0,85$ (порівняно з 0,55 і 0,50 для TF-IDF). Підвищені порогови не свідчать про вищу якість, а лише відображають відмінність шкал двох методів.

4.3.6 Ансамблевий метод верифікації

TF-IDF та семантичної верифікації фіксують різні аспекти семантичної близькості: перший – лексичний збіг ключових термінів, другий – семантичну спорідненість на рівні розподілених представлень. Природним подальшим кроком є побудова ансамблевого методу, що комбінує обидва сигнали в єдину оцінку, поєднуючи переваги кожного підходу.

У роботі обрано просту лінійну комбінацію оцінок за формулою (4.13):

$$score_ensemble = w_{tfidf} \cdot score_tfidf + w_{semantic} \cdot score_semantic, \quad (4.13)$$

де w_{tfidf} та $w_{semantic}$ – невід’ємні вагові коефіцієнти з обмеженням $w_{tfidf} + w_{semantic} = 1$.

Перед об’єднанням оцінки приводяться до спільної шкали $[0; 1]$ за допомогою min-max нормалізації за пакетом всіх кандидатів, що дозволяє коректно комбінувати різномірні значення.

Альтернативні стратегії – мажоритарне голосування на рівні класів, стекінг через мета-класифікатор, або підбір порогів окремо для кожного методу з подальшим логічним «АБО» – розглядалися, проте відкинуті як такі, що або потребують навчальної вибірки (стекінг), або втрачають інформацію про градації подібності (мажоритарне голосування на класах).

Оптимальні значення w_{tfidf} та $w_{semantic}$ визначаються через двовимірний grid search на просторі $(w_{tfidf}, w_{semantic})$ за умови $w_{tfidf} + w_{semantic} = 1$, з кроком 0,1. Цільовою функцією є макроусереднена міра F1 (macro-F1) на розміченому наборі даних. Експериментально (див. підрозділ 5.4) оптимальною конфігурацією

виявилося поєднання $w_{tfidf} = 0,9$ та $w_{semantic} = 0,1$, що свідчить про істотне домінування TF-IDF верифікатора в задачі для україномовних коротких новинних повідомлень.

Несподіваним, але стійким результатом є саме мала вагомість семантичного компонента в ансамблі. Це пояснюється кількома факторами:

- повідомлення в Telegram-каналах, як правило, мають короткий обсяг (1–3 речення), що обмежує переваги нейромережевого кодування – короткі тексти не дають MiniLM достатнього контексту для розкриття її переваг над лексичним підходом;
- морфологічна лематизація `ru morphology3` на етапі попередньої обробки вже компенсує більшу частину поверхневої лексичної варіативності української мови, наближаючи якість TF-IDF до якості семантичного методу;
- модель `paraphrase-multilingual-MiniLM-L12-v2` не була спеціально натренована на україномовних даних – її україномовна якість істотно нижча, ніж англомова;
- у задачі верифікації новин лексичний збіг є надійнішим сигналом, ніж семантична спорідненість: дві новини про різні події можуть бути семантично близькими (обстріли, бойові дії, політичні заяви), але саме збіг ключових імен, місць та чисел свідчить про опис тієї ж події.

Для ансамблевого методу застосовується той самий двопороговий механізм (підрозділ 4.3.3), але з порогоми, оптимізованими саме для розподілу *score_ensemble*. За результатами `grid search` оптимальними виявилися значення $\tau_{verified} = 0,55$ та $\tau_{uncertain} = 0,50$, що збігаються з порогоми TF-IDF – це додатково підтверджує домінування TF-IDF компонента в ансамблі.

Незважаючи на формальну перевагу ансамблю в окремих метриках (див. підрозділ 5.4), його практичне використання обмежене: семантичний компонент потребує завантаження трансформерної моделі обсягом близько 400 МБ, забезпечення обчислювального ресурсу для інференсу (бажано GPU або сучасний CPU) і, як наслідок, на 1–2 порядки повільніший за TF-IDF. У ситуаціях,

коли важлива швидкодія в режимі реального часу або обмежені обчислювальні ресурси, рекомендується застосовувати TF-IDF верифікатор як самостійний метод. Ансамбль доцільно застосовувати в режимі офлайн-обробки великих архівів, де якість важливіша за швидкість.

4.4 Проєктування архітектури програмного забезпечення

Для реалізації розроблених алгоритмів було спроектовано модульну архітектуру системи, яка базується на принципах слабкої зв'язності (low coupling) компонентів. Такий підхід дозволяє незалежно розробляти, тестувати та масштабувати окремі частини системи, забезпечуючи гнучкість при зміні джерел даних або вдосконаленні алгоритмів верифікації без впливу на загальну структуру.

Архітектура системи ієрархічно розділена на чотири основні рівні: рівень збору даних (Data Collection Layer), рівень обробки даних (Data Processing Layer), рівень зберігання (Data Storage Layer) та рівень аналізу і верифікації (Analysis & Verification Layer), причому останній складається з двох незалежних верифікаторів та модуля ансамблевого комбінування. Така сегрегація відповідальності дозволяє чітко розмежувати етапи життєвого циклу даних від моменту їх отримання до фінальної класифікації.

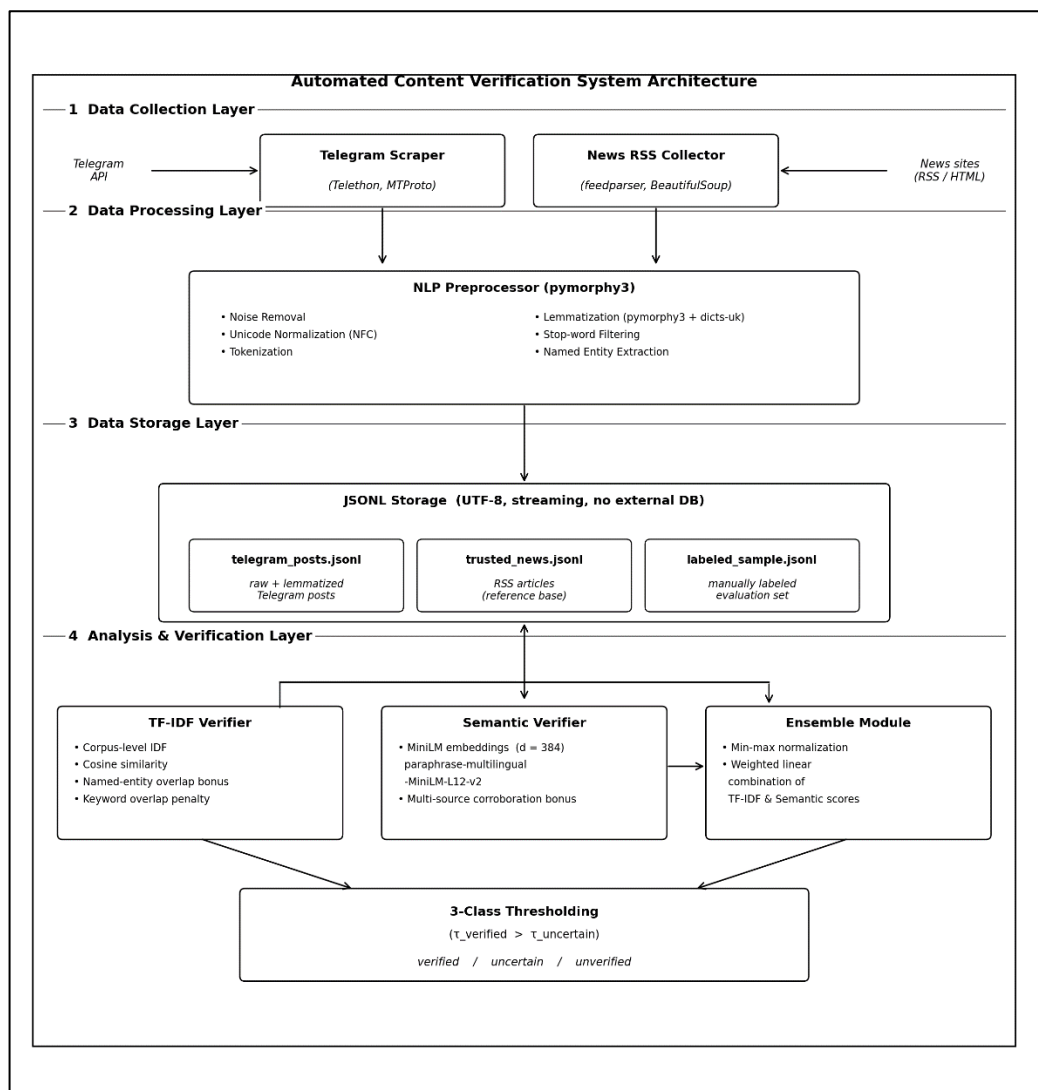


Рисунок 4.1 – Архітектура системи автоматичної верифікації контенту (рисунок створено самостійно)

Зазначимо, що блок «Аналіз і верифікація» на рис. 4.1 узагальнено представляє три різні реалізовані компоненти – TF-IDF верифікатор, семантичний верифікатор і ансамблевий модуль (підрозділи 4.3.4–4.3.6), які функціонують паралельно та незалежно один від одного, а їх вихідні оцінки агрегуються ансамблевим модулем для прийняття остаточного рішення.

Розроблена архітектура, представлена на рис. 4.1, забезпечує конвеєрну обробку інформації. Модулі збору даних (Telegram Scraper та News RSS Collector) наповнюють файли JSONL через проміжний етап NLP-обробки (очищення, лематизація, виділення сутностей). Далі два незалежні верифікатори – TF-IDF та семантичний – оцінюють кожне повідомлення з Telegram у порівнянні з новинами-

кандидатами з еталонної бази. Ансамблевий модуль агрегує їхні оцінки з ваговими коефіцієнтами та виносить остаточне рішення про присвоєння одного з трьох класів – *verified*, *uncertain* або *unverified*.

На рівні збору даних функціонують два ключові компоненти, що працюють паралельно. Перший – це *Telegram Scraper*, реалізований на базі асинхронної бібліотеки *Telethon*. Він виконує авторизацію через протокол *MTProto*, що дозволяє емулювати дії реального користувача, моніторити визначений список каналів та завантажувати нові повідомлення в режимі реального часу з мінімальною затримкою. Другим компонентом є *News RSS Collector*, який використовує бібліотеку *feedparser* для періодичного обходу RSS-стрічок довірених новинних агентств (зокрема «Українська правда», «Суспільне» та інших). Для випадків, коли RSS-стрічка містить лише анонс, для отримання повного тексту статті використовуються бібліотеки *requests* та *BeautifulSoup* [18]. Задача компоненту – вилучення заголовків, основного тексту новин та метаданих (час публікації, посилання, джерело) для формування актуальної еталонної бази знань, з якою будуть порівнюватися повідомлення з месенджерів. Вибір RSS як основного протоколу обумовлений його стабільністю, наявністю на всіх провідних новинних ресурсах та простотою порівняно зі скрапінгом HTML, схема якого може довільно змінюватися.

Рівень обробки даних виступає проміжною ланкою, де відбувається трансформація «сирих» даних у структурований формат. Центральним елементом тут є модуль *Preprocessor*, який реалізує NLP-пайплайн, детально описаний у попередніх підрозділах. Він відповідає за очищення тексту від специфічного шуму (емодзі, URL-посилань, @згадок та хештегів), Unicode-нормалізацію (NFC), морфологічну лематизацію засобами *rumorphy3*, фільтрацію стоп-слів (українських, російських, англійських) та виділення іменованих сутностей (через розпізнавання слів із великої літери та повністю великих аббревіатур). Саме на цьому етапі неструктурований текст перетворюється на нормалізований набір лем разом із множиною ключових іменованих сутностей, придатний для математичного моделювання.

Для рівня зберігання даних обрано формат JSONL (JSON Lines, UTF-8) – потоковий текстовий формат, у якому кожен рядок файлу містить незалежний JSON-об'єкт. Дані організовано у вигляді кількох файлів за функціональним призначенням: `telegram_posts.jsonl` зберігає оригінальні повідомлення з Telegram разом з їхніми лематизованими версіями, метаданими (дата, канал, кількість переглядів) та результатами верифікації; `trusted_news.jsonl` містить перевірені новини з офіційних джерел, які слугують еталоном для порівняння; `labeled_sample.jsonl` містить розмічену людиною вибірку для оцінювання моделей. Вибір JSONL замість повноцінної СУБД обґрунтований специфікою дослідницької роботи: відсутність зовнішніх залежностей спрощує відтворюваність експериментів, потокова обробка зменшує споживання пам'яті, а текстова природа формату дозволяє переглядати, версіювати та обмінюватися даними стандартними засобами (Git, diff, текстові редактори). Pydantic-моделі забезпечують валідацію структури даних на вході та виході, компенсуючи відсутність схеми на рівні сховища.

Ядром системи є рівень аналізу та верифікації, де реалізовано три способи оцінювання семантичної близькості. TF-IDF Verifier використовує статистичну векторизацію з корпусно-рівневою IDF та обчислює косинусну подібність із бонусом за збіг іменованих сутностей. Semantic Verifier застосовує багатомовну трансформерну модель `paraphrase-multilingual-MiniLM-L12-v2` для отримання щільних семантичних векторів і обчислює їхню косинусну подібність із бонусом за корроборацію декількома джерелами. Ensemble Module лінійно комбінує оцінки обох верифікаторів із оптимальними ваговими коефіцієнтами (0,9 для TF-IDF та 0,1 для семантичного – за результатами `grid search`). Кожен верифікатор виконує попередню фільтрацію кандидатів за часовим вікном, що оптимізує обчислювальні витрати. Рішення про присвоєння одного з трьох класів (`verified` / `uncertain` / `unverified`) виноситься на основі двох порогових значень.

Взаємодія компонентів відбувається за чітко визначеним сценарієм: Telegram Scraper отримує нове повідомлення та передає його в Preprocessor. Після очищення та лематизації текст серіалізується в JSONL-файл `telegram_posts.jsonl`, що

автоматично ініціює процес верифікації. Verifier зчитує релевантні новини-кандидати з `trusted_news.jsonl` у часовому вікні, проводить порівняльний аналіз із застосуванням обох методів (TF-IDF та семантичного), агрегує результати ансамблевим модулем і дописує оновлений запис у JSONL-файл із полями `verification_status`, `similarity_score` та посиланням на знайдене першоджерело. Така архітектура забезпечує високу швидкодію та надійність системи, дозволяючи обробляти великі потоки інформації в автоматичному режимі без потреби в зовнішніх сервісах.

5 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ

Метою експериментального дослідження є оцінка ефективності розроблених методів автоматичної верифікації повідомлень з Telegram-каналів. Дослідження проведено на власноруч зібраному та розміченому наборі даних і охоплює три розроблені методи (TF-IDF верифікатор, семантичний верифікатор та ансамблевий метод) у двох конфігураціях кожен – базовій (без введених удосконалень) та фінальній (з усіма емпірично оптимальними параметрами). Експеримент включає пошук гіперпараметрів за допомогою grid search, обчислення стандартних метрик класифікації, побудову матриці похибок та якісний аналіз помилок.

5.1 Набір даних та процес анотації

Збір даних здійснено з двох типів джерел протягом двох календарних місяців (березень-квітень 2026 р.). Перший тип – три україномовні новинні Telegram-канали різного профілю, що дозволило отримати лексичне різноманіття: від офіційних повідомлень державних структур до неформальної подачі новин блогерами. Загалом за період збору з трьох каналів зібрано 1042 текстових повідомлення довжиною від 5 до 800 слів (медіана – 32 слова). Технічно збір реалізовано засобами бібліотеки Telethon через протокол MTProto, що дозволило отримати повідомлення разом із метаданими (час публікації, ідентифікатор каналу, кількість переглядів, посилання на пересилання).

Другий тип – RSS-стрічки авторитетних українських новинних ресурсів («Українська правда», «Суспільне» та інші). Збір новин з RSS-стрічок здійснювався безперервно протягом усього періоду досліджень засобами бібліотеки feedparser. Для випадків, коли RSS-стрічка містила лише анонс новини, повний текст статті отримувався додатковим HTTP-запитом із подальшим парсингом HTML за допомогою бібліотеки BeautifulSoup. Сформована еталонна база знань містить заголовки, основний текст і час публікації кожної новини.

Важливою методологічною умовою збору було узгодження часових зрізів двох наборів даних: для кожного повідомлення з Telegram у момент аналізу

обиралися лише ті новинні статті, які знаходяться в часовому вікні ± 48 годин від моменту його публікації (підрозділ 4.3.1). Це моделює реалістичний сценарій онлайн-верифікації: системі недоступні майбутні новини.

Для проведення кількісної оцінки розроблених методів необхідна вибірка повідомлень з Telegram, для яких вручну визначено справжній клас. Через високу трудомісткість ручної розмітки (анотатор має ознайомитися як з повідомленням, так і з усіма потенційними новинами-першоджерелами) повна розмітка усіх 1042 повідомлень виявилась нераціональною. Натомість сформовано стратифіковану підвибірку обсягом 305 повідомлень.

Стратифікація здійснювалася за діапазонами TF-IDF оцінки подібності, отриманої базовим методом. Усю вихідну множину з 1042 повідомлень було розбито на 5 страт за значенням *base_score*: [0,0; 0,1), [0,1; 0,3), [0,3; 0,5), [0,5; 0,7), [0,7; 1,0]. З кожної страти випадковим чином відібрано пропорційну кількість зразків. Така схема забезпечила достатнє покриття як «легких» зразків (явно *verified* або явно *unverified*, що мають екстремальні значення подібності), так і граничних випадків – найбільш цікавих з методичної точки зору, оскільки саме на них перевіряється здатність системи до точного прийняття рішень.

Для кожного повідомлення з підвибірки анотатором (автором роботи) визначався один з трьох класів. Анотатору надавалися: текст повідомлення з Telegram, його метадані (канал, час), а також автоматично знайдена топ-десятка новин-кандидатів з еталонної бази у часовому вікні ± 48 годин. Анотатор переглядав запропонованих кандидатів і за необхідності здійснював додатковий пошук у відкритих джерелах для повного розуміння контексту події.

Класи визначалися за наступною схемою:

- *verified*: повідомлення з Telegram описує подію, яка повністю висвітлена принаймні одним з кандидатів у часовому вікні. Подія, її учасники, місце та час підтверджуються авторитетним джерелом;
- *uncertain*: повідомлення описує подію, яка частково підтверджується кандидатами: подія справді мала місце, але деталі (точна цифра, формулювання, конкретні наслідки) розходяться, або висвітлення в

- авторитетних джерелах є неповним. Цей клас принципово відрізняється від оцінки впевненості системи – він відображає реальну ситуацію часткового покриття, що часто виникає в оперативних новинних потоках;
- **unverified**: у часовому вікні немає кандидатів, які підтверджують подію. Це може свідчити про дезінформацію, надмірно локальну подію (нерелевантну для авторитетних ЗМІ), особисту думку автора каналу або фундаментальне обмеження методу для надто стислих повідомлень.

Розподіл класів у розміченій вибірці характеризується істотною незбалансованістю та представлений на рис. 5.1 та в табл. 5.1.

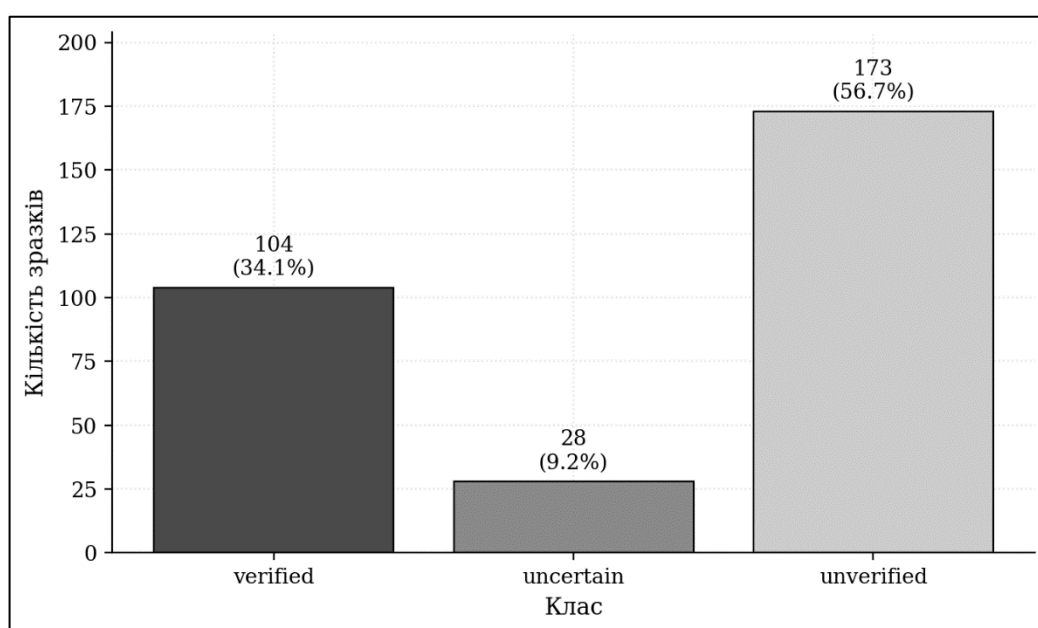


Рисунок 5.1 – Розподіл класів у розміченій вибірці (рисунок створено самостійно)

Таблиця 5.1 – Розподіл класів у розміченій вибірці (всього 305 зразків)

Клас	Кількість зразків	Частка, %
verified	104	34,1
uncertain	28	9,2
unverified	173	56,7
Усього	305	100,0

Як видно з табл. 5.1, клас *unverified* є домінантним (56,7 %), що відповідає інтуїтивному очікуванню: значна частина повідомлень в Telegram-каналах не має прямого еквіваленту в авторитетних новинних ЗМІ – це або суб’єктивні коментарі, або занадто оперативна інформація, що не встигає потрапити в редакційний цикл. Натомість клас *uncertain* є помітно недопредставленим (лише 9,2 %), що в подальшому зумовлює методичні труднощі при оцінці саме цього класу.

Слід чесно визнати кілька методологічних обмежень розміченої вибірки:

- один анотатор. Розмітку виконав одноосібно автор роботи, що не дозволяє провести вимірювання міжанотаторської узгодженості (*inter-annotator agreement*, наприклад через коефіцієнт Cohen’s κ). Для повноцінного академічного дослідження бажано залучення щонайменше двох незалежних анотаторів – це є одним з очевидних напрямів подальшого розвитку дослідження;
- можливі суб’єктивні викривлення в граничних випадках. Розрізнення між класами *verified* та *uncertain* не завжди є очевидним; межа залежить від тлумачення анотатором того, що вважати «повним» покриттям. Стратифікована вибірка, описана у попередньому підрозділі, частково компенсує цей фактор, забезпечуючи достатнє охоплення граничних випадків;
- обмежений обсяг класу *uncertain*. Лише 28 зразків класу *uncertain* є недостатньою основою для надійних статистичних висновків щодо роботи методу саме на цьому класі. Це зумовлює ширші довірчі інтервали для відповідних метрик і пояснює нижчу якість моделей саме на цьому класі (див. підрозділ 5.5).

5.2 Метрики оцінювання ефективності

Для оцінювання якості розроблених методів використано стандартні метрики задачі багатокласової класифікації. Кожна з метрик характеризує роботу системи з певної сторони, і лише їх комплексне застосування дозволяє сформулювати об’єктивне уявлення про якість методу.

Для кожного з трьох класів $c \in \{\text{verified}, \text{uncertain}, \text{unverified}\}$ обчислюються три класичні метрики.

Precision (точність) – частка зразків, правильно класифікованих у клас c , серед усіх зразків, для яких система передбачила цей клас, згідно з формулою (5.1):

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad (5.1)$$

де TP_c – справжньо-позитивні передбачення;

FP_c – хибнопозитивні.

Recall (повнота) – частка правильно класифікованих зразків класу c серед усіх зразків цього класу в розміненій вибірці, згідно з формулою (5.2):

$$R_c = \frac{TP_c}{TP_c + FN_c}, \quad (5.2)$$

де FN_c – хибнонегативні передбачення.

F1-score – гармонічне середнє Precision та Recall за формулою (5.3):

$$F1_c = \frac{2 \cdot P_c \cdot R_c}{P_c + R_c}. \quad (5.3)$$

F1 збалансовує точність та повноту і є найважливішою метрикою на рівні окремого класу.

На рівні всієї системи використовуються:

Accuracy (загальна точність) – частка правильно класифікованих зразків за формулою (5.4):

$$\text{Accuracy} = \frac{1}{N} \sum_c TP_c, \quad (5.4)$$

де N – загальна кількість зразків.

Через істотну незбалансованість класів (56,7 % *unverified*) *accuracy* сама по собі не є надійним показником: тривіальний класифікатор, що завжди передбачає *unverified*, отримав би *accuracy* 56,7 %. Тому *accuracy* використовується як допоміжна метрика.

Macro-F1 – незважене усереднення F1 за всіма трьома класами, відповідно формула (5.5):

$$\text{Macro-F1} = \frac{F1_{\text{verified}} + F1_{\text{uncertain}} + F1_{\text{unverified}}}{3}. \quad (5.5)$$

Саме *macro-F1* обрано як основну оптимізаційну метрику. Незважена усередненість гарантує, що жоден з трьох класів не може бути «принесений у жертву» для покращення сумарного показника. Зокрема, *macro-F1* справедливо «штрафує» за погану якість роботи на меншості класів: *uncertain* (9,2 %) та *verified* (34,1 %), що відповідає практичним вимогам: для побудови верифікованого корпусу однаково важливо як знаходити підтверджені новини, так і коректно ідентифікувати непідтверджені.

Binary Verified F1 – F1-міра в режимі бінарної класифікації, де *verified* та *uncertain* об'єднуються в позитивний клас, а *unverified* є негативним. Ця метрика зручна для порівняння з традиційними бінарними системами фактчекінгу та для практичного оцінювання «знайдено хоч щось» сценарію.

Зведені метрики не дозволяють виявити характерні шаблони помилок системи. Для якісного аналізу будується матриця похибок (*confusion matrix*) – таблиця 3×3 , де рядки відповідають справжнім класам, а стовпці – передбаченим. Елемент (i, j) показує кількість зразків справжнього класу i , класифікованих системою як j . Діагональні елементи відповідають правильним передбаченням, а недіагональні відповідають помилкам, причому положення помилок у матриці дозволяє ідентифікувати, які класи система найчастіше плутає між собою.

5.3 Пошук гіперпараметрів (Grid Search)

Розроблені методи містять кілька гіперпараметрів, оптимальні значення яких неможливо обрати аналітично. Їх необхідно визначати емпірично на розміченій вибірці. Для пошуку оптимуму застосовано техніку *grid search* – вичерпного пошуку у просторі параметрів із заданим кроком. Цільовою функцією для всіх запусків *grid search* є *macro-F1*.

Загалом для трьох методів виконано чотири окремих процедури *grid search*, реалізовані як окремі скрипти на мові Python:

- *Grid search* для TF-IDF: пошук у двовимірному просторі ($\tau_{verified} \times \tau_{uncertain}$) з кроком 0,05 у діапазоні [0,30; 0,70];
- *Grid search* для семантичного методу: аналогічна структура, але з адаптованими діапазонами [0,75; 0,99] для $\tau_{verified}$ та [0,65; 0,95] для $\tau_{uncertain}$ з кроком 0,05;
- *Grid search* для ансамблю: тривимірний простір ($w_{tfidf} \times w_{semantic} \times \tau$), причому через обмеження $w_{tfidf} + w_{semantic} = 1$ фактично двовимірний. Ваги перебиралися з кроком 0,1, пороги – з кроком 0,05;
- *Grid search* для коефіцієнтів бонусів – окрема процедура для тонкого налаштування коефіцієнтів α (бонус за іменовані сутності) та β (бонус за корроборацію) на основі попередньо обчислених проміжних оцінок (без перерахунку основного pipeline), що значно економить обчислювальні ресурси.

Для всіх процедур передбачено автоматичну фільтрацію некоректних комбінацій (зокрема, неприпустима умова $\tau_{uncertain} > \tau_{verified}$).

Найважливіший *grid search* – це пошук оптимальних порогів для TF-IDF верифікатора, який наочно представлений на рис. 5.2. Heatmap показує значення *macro-F1* для кожної припустимої комбінації порогів.

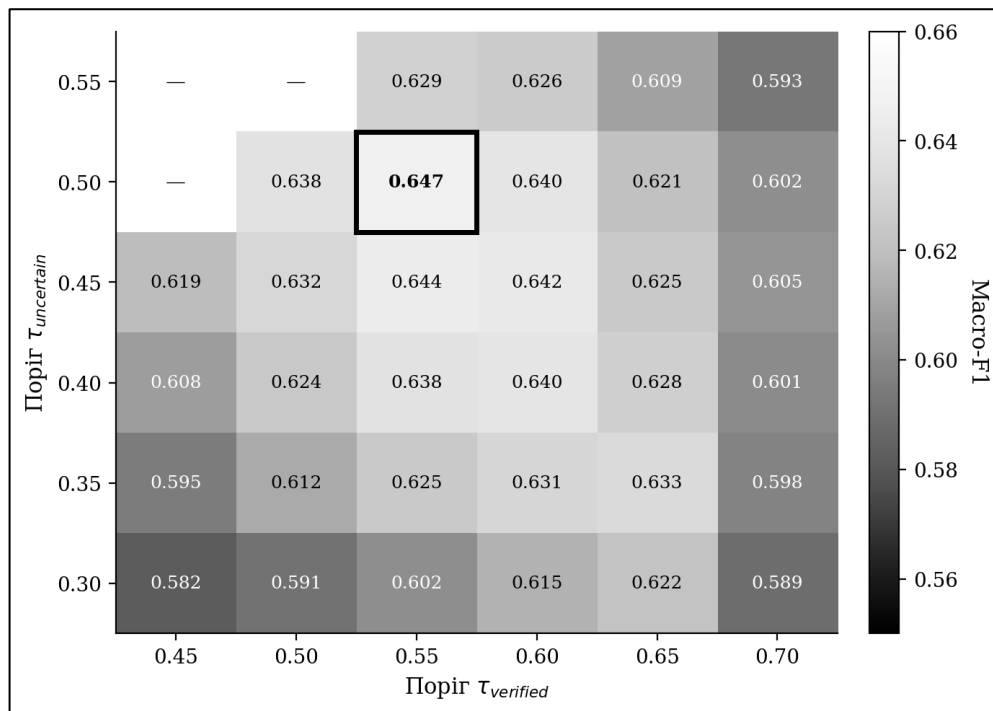


Рисунок 5.2 – Heatmap значень macro-F1 у просторі $(\tau_{verified} \times \tau_{uncertain})$ для TF-IDF верифікатора (рисунок створено самостійно)

Як видно з рис. 5.2, поверхня цільової функції є гладкою з виразним глобальним оптимумом у точці $(\tau_{verified} = 0,55; \tau_{uncertain} = 0,50)$, де досягається $\text{macro-F1} = 0,647$. Пробіли в нижньому-правому трикутнику матриці відповідають недопустимим комбінаціям. Відносно повільне спадання в околі оптимуму свідчить про стійкість методу до невеликих змін гіперпараметрів – це позитивна властивість, що означає неперенавчання на розмічену вибірку.

Окрему увагу приділено визначенню оптимального коефіцієнта α бонусу за збіг іменованих сутностей (підрозділ 4.3.4). Залежність метрик від α представлена на рис. 5.3.

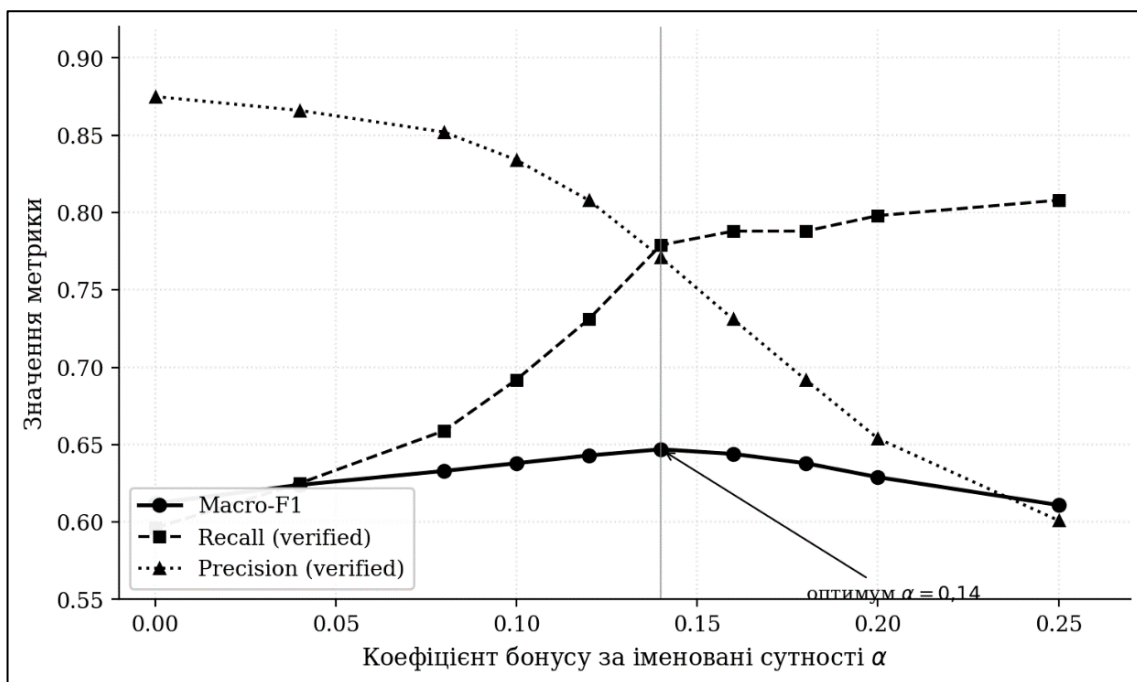


Рисунок 5.3 – Залежність метрик якості TF-IDF верифікатора від коефіцієнта бонусу за іменовані сутності α (рисунок створено самостійно)

З рис. 5.3 видно характерний компроміс між Precision та Recall: збільшення α монотонно підвищує Recall класу verified (з 59,6 % при $\alpha = 0$ до 80,8 % при $\alpha = 0,25$), але водночас знижує Precision (з 87,5 % до 60,1 %). Macro-F1, що збалансовує обидві метрики, досягає максимуму 0,647 при $\alpha = 0,14$, після чого починає зменшуватися. Саме це значення α і прийнято як фінальне для TF-IDF верифікатора. Рисунок наочно ілюструє ключову методологічну тезу: при налаштуванні системи неможливо одночасно максимізувати точність і повноту, і вибір оптимуму залежить від практичної цілі. У контексті задачі побудови корпусу пропуск verified-новини є гіршою помилкою, ніж включення помилкового кандидата (який все одно може бути перевірений на наступних етапах), тому зміщення в бік вищого Recall є виправданим. Таблиця 5.2 узагальнює результати всіх процедур grid search.

Таблиця 5.2 – Оптимальні значення гіперпараметрів за результатами grid search.

Параметр	TF-IDF	Семантичний	Ансамбль
$\tau_{verified}$	0,55	0,95	0,55
$\tau_{uncertain}$	0,50	0,85	0,50
α (бонус за сутності)	0,14	0,04	—
β (бонус за корроборацію)	0,00	0,07	—
w_{tfidf}	—	—	0,90
$w_{semantic}$	—	—	0,10
Часове вікно, год	48	48	48
<i>MIN_KEYWORD_OVERLAP</i>	2	—	2
Топ- <i>K</i> кандидатів (semantic)	—	5	5

Особливо цікавий результат: оптимальне значення $\beta = 0,00$ для TF-IDF, що означає, що бонус за корроборацію декількома джерелами в TF-IDF верифікаторі не лише не корисний, а й шкідливий. Емпіричне обґрунтування цього факту наведено в підрозділі 4.3.5.

5.4 Результати та порівняльний аналіз методів

Після підбору оптимальних гіперпараметрів проведено фінальне оцінювання всіх трьох методів на розміченій вибірці з 305 зразків. Оцінювання здійснено для двох конфігурацій кожного методу: базової – без застосування описаних удосконалень (одиничний поріг, без бонусів, без корпусно-рівневої IDF) та фінальної – з усіма емпірично оптимальними параметрами. Результати наведено в табл. 5.3 та проілюстровані на рис. 5.4.

Таблиця 5.3 – Порівняльна характеристика методів верифікації (всі значення у %)

	Метод	Конфігурація	Асс.	$F1_{\text{ver.}}$	$F1_{\text{unc.}}$	$F1_{\text{unv.}}$	Macro-F1
1	TF-IDF	базова	78,0	71,7	38,5	86,2	65,5
2	TF-IDF	фінальна	81,0	77,5	26,7	89,9	64,7
3	Semantic	базова	74,6	64,2	16,7	86,0	55,6
4	Semantic	фінальна	73,1	69,3	24,1	83,3	58,9
5	Ensemble	базова	81,7	71,7	41,2	88,8	67,2
6	Ensemble	фінальна	80,7	79,5	25,5	89,0	64,6

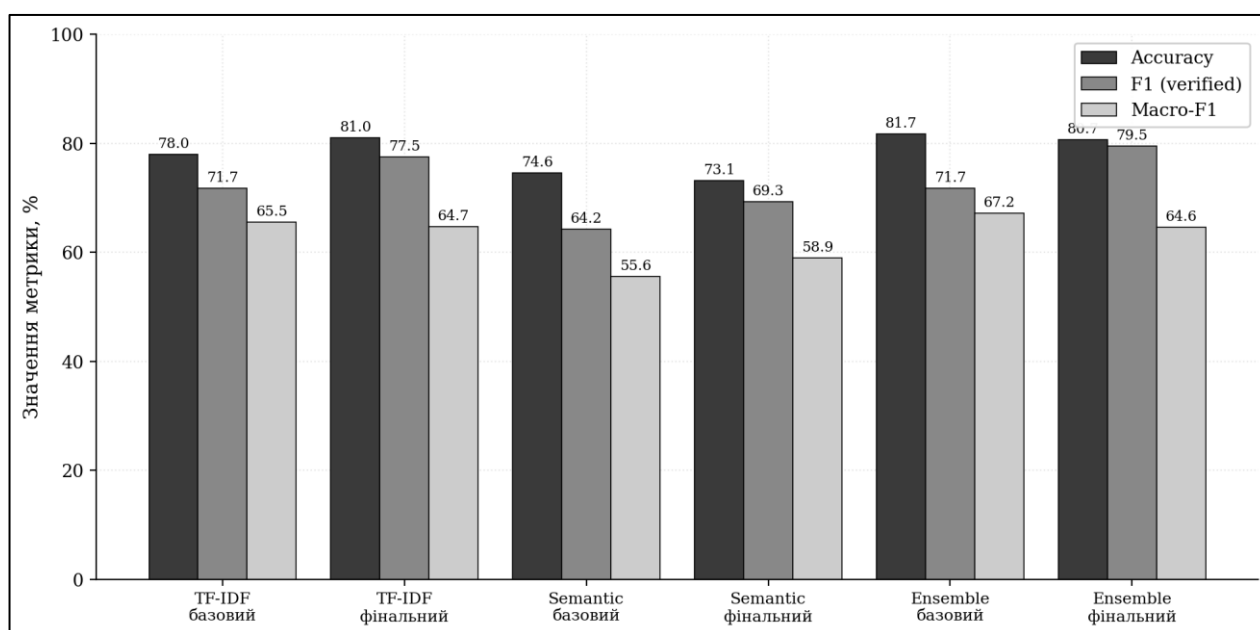


Рисунок 5.4 – Порівняння методів верифікації за основними метриками (рисунок створено самостійно)

З наведених у табл. 5.3 та на рис. 5.4 даних можна зробити декілька принципово важливих висновків.

Перевага TF-IDF підходу. Для україномовних коротких новинних повідомлень саме TF-IDF верифікатор з удосконаленнями (рядок 2 табл. 5.3) виявляється найкращим за критерієм відношення «якість/обчислювальна складність». Він досягає Ассигасу = 81,0 % та $F1_{\text{verified}} = 0,775$ – другий за всіма метриками результат серед фінальних конфігурацій, при цьому в десятки разів

швидший за семантичні методи й не потребує завантаження 400-мегабайтної моделі.

Несподівана слабкість семантичного методу. Семантичний верифікатор показує помітно нижчі результати, ніж TF-IDF: $\text{macro-F1} = 0,589$ проти $0,647$. Цей результат на перший погляд парадоксальний, оскільки нейромережеві моделі зазвичай переважають класичні статистичні підходи. Проте поглиблений аналіз пояснює явище:

- модель `paraphrase-multilingual-MiniLM-L12-v2` не була спеціально натренована на україномовних даних, тому її якість для української мови помітно нижча від англійської базової лінії;
- повідомлення з Telegram є переважно короткими (медіана 32 слова), а семантичні моделі найкраще працюють на текстах середньої та великої довжини, де є достатній контекст;
- морфологічна лематизація `ru morphology` на етапі попередньої обробки вже компенсує більшу частину поверхневої лексичної варіативності, що зменшує перевагу нейромережевого підходу.

Граничне поліпшення ансамбля. Ансамблеві методи (рядки 5–6) формально показують найвищі результати: $\text{Assurasy} = 81,7\%$ для базової конфігурації, але різниця з найкращим TF-IDF ($81,0\%$) не перевищує 1 процентний пункт. Оптимальні ваги ансамблю $w_{tfidf} = 0,9/w_{semantic} = 0,1$ додатково підтверджують, що TF-IDF домінує і в комбінованому методі. Враховуючи, що ансамбль вимагає обчислення обох незалежних оцінок (з відповідною обчислювальною вартістю), його практична доцільність є обмеженою.

Вплив удосконалень. Порівняння базових (рядки 1, 3, 5) і фінальних (рядки 2, 4, 6) конфігурацій демонструє важливість запропонованих удосконалень. Для TF-IDF: підвищення assurasy з $78,0\%$ до $81,0\%$ і $F1_{\text{verified}}$ з $0,717$ до $0,775$ при незмінному macro-F1 – це означає, що бонус за іменовані сутності значно покращує верифікацію класу `verified` ціною незначних втрат для класу `uncertain`.

Для якісного розуміння природи прийняття рішень розроблений метод TF-IDF подано на рис. 5.5 у вигляді скрипичних діаграм (violinplots) розподілу *final_score* за справжніми класами.

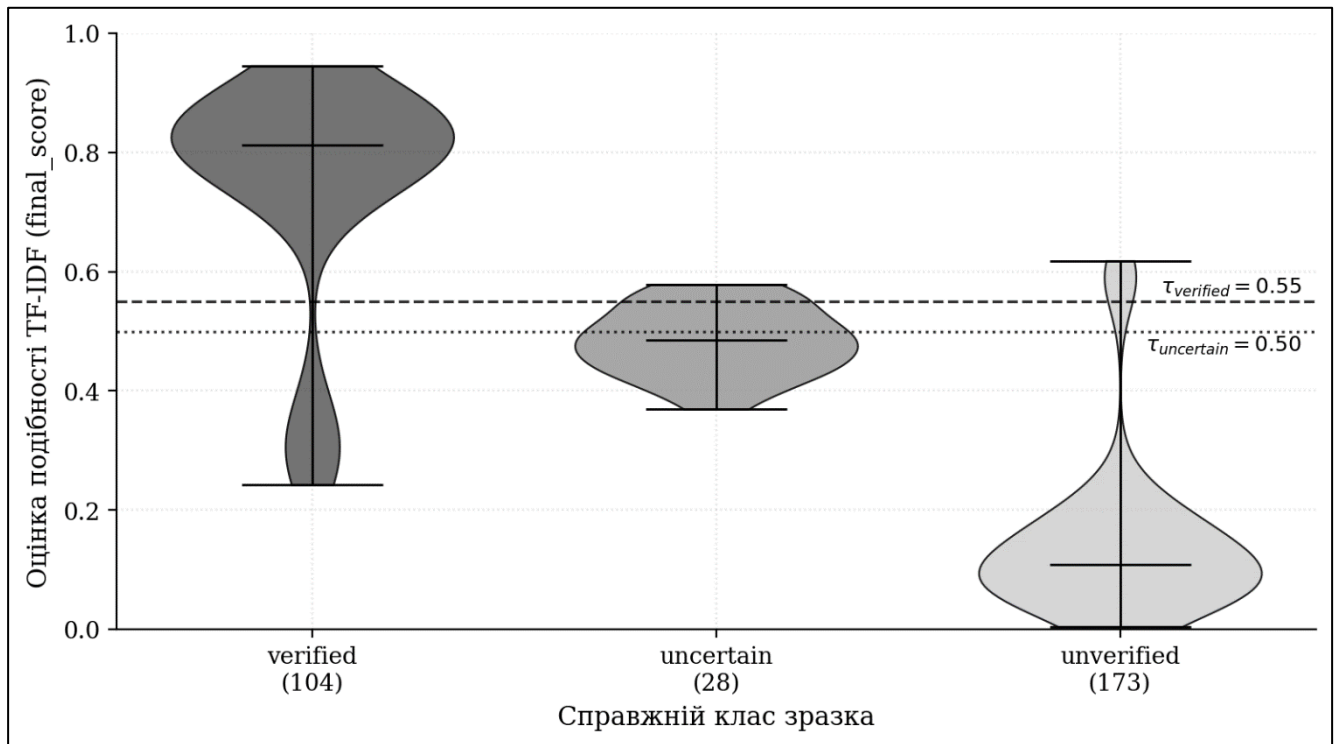


Рисунок 5.5 – Розподіл фінальних оцінок подібності TF-IDF за справжніми класами зразків (рисунок створено самостійно)

- З рис. 5.5 видно, що розподіл оцінок суттєво відрізняється для трьох класів:
- клас *verified* має бімодальний розподіл: переважна більшість зразків концентрується в діапазоні $[0,7; 0,95]$ (правильно класифікуються як *verified*), проте є помітний «хвіст» у діапазоні $[0,2; 0,3]$, що відповідає тим 16 зразкам, які система помилково класифікує як *unverified* (детальний аналіз у підрозділі 5.5);
 - клас *uncertain* зосереджений саме у вузькому діапазоні $[0,38; 0,55]$ між двома порогами, що підтверджує адекватність обраної 3-класової схеми – ці зразки справді знаходяться «посередині»;

- клас *unverified* має одномодальний розподіл, концентрований у діапазоні $[0,0; 0,2]$, причому розподіл швидко спадає вище 0,3, що свідчить про надійне розрізнення цього класу від двох інших.

Особливо вартим уваги є вузька й чітка концентрація класу *uncertain* саме в проміжній зоні. Цей факт є емпіричним підтвердженням принципової змістової відмінності класу *uncertain* від невпевненості системи: якби *uncertain* був лише «межовим» класом, його розподіл був би розмазаний; натомість, як видно з рис. 5.5, цей клас має власний характерний «острівець» на скрипичній діаграмі.

Для основного фінального методу (TF-IDF з удосконаленнями) матриця похибок представлена на рис. 5.6.

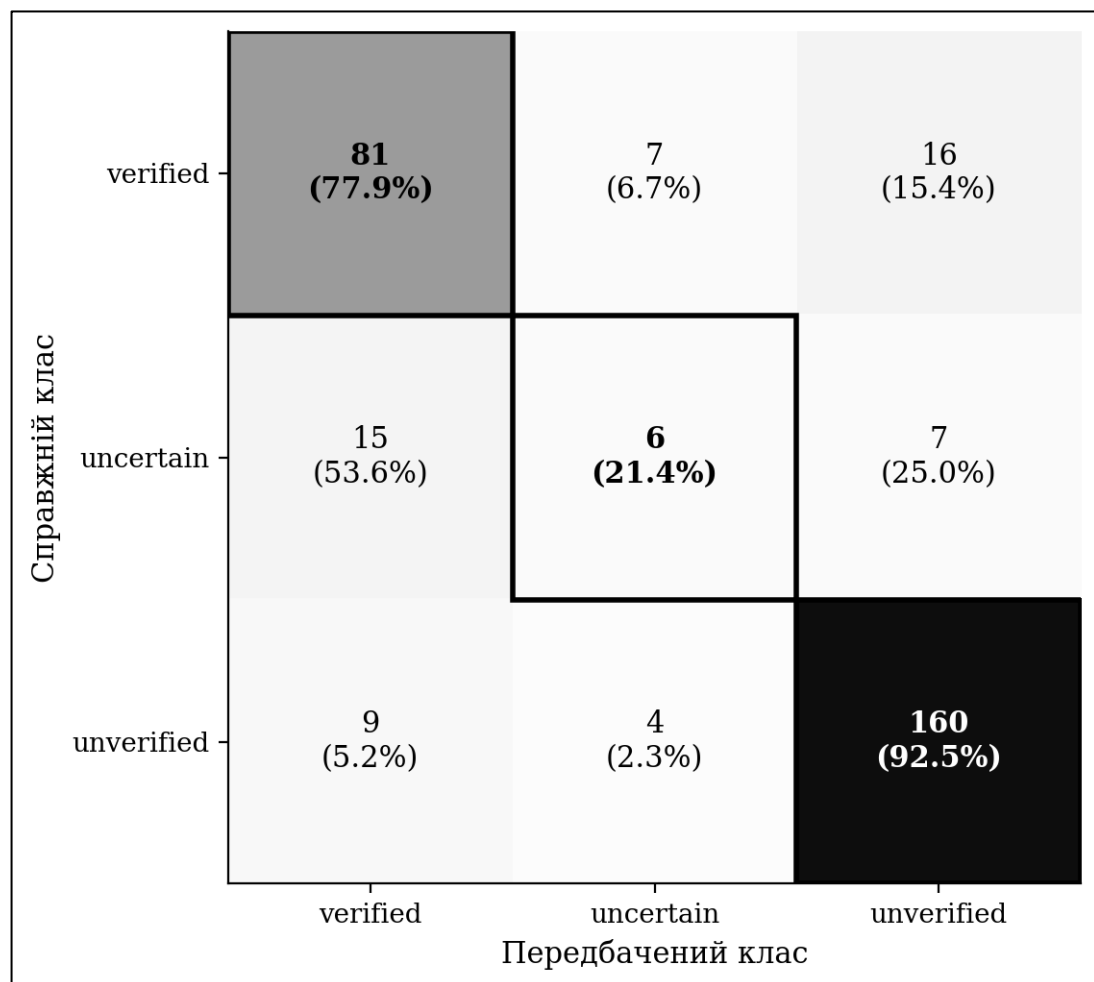


Рисунок 5.6 – Матриця похибок TF-IDF верифікатора (рисунок створено самостійно)

Матриця похибок дозволяє якісно проаналізувати поведінку системи. По-перше, діагональні елементи (виділені рамками) демонструють хороше розпізнавання основних класів: 81 з 104 справжніх *verified* (77,9 %), 6 з 28 справжніх *uncertain* (21,4 %), 160 з 173 справжніх *unverified* (92,5 %). По-друге, недіагональні елементи свідчать про характерні шаблони помилок:

- помилки на класі *verified*. З 23 неправильно класифікованих *verified*-зразків 16 потрапили в *unverified* (істотніша помилка) і 7 – в *uncertain* (м'якша помилка, оскільки *uncertain* все ще зберігає посилання на знайдене першоджерело). Це означає, що з точки зору практичного використання корпусу, втрачено 16 справжніх верифікованих новин;
- помилки на класі *uncertain*. Найбільша частка помилок на цьому класі: 22 з 28 зразків неправильно класифіковано (78,6 %). Розподіл помилок: 15 – у *verified*, 7 – у *unverified*. Більшість зразків *uncertain* система схильна вважати *verified*, що пояснюється бонусом за іменовані сутності, який «штовхає» граничні зразки догори;
- помилки на класі *unverified*. Лише 13 з 173 зразків (7,5 %) неправильно класифіковано. З них 9 потрапили в *verified* – це хибнопозитивні спрацювання, які знижують Precision класу *verified* до 77,1 %.

5.5 Аналіз помилок та обмежень

Поглиблений якісний аналіз 16 зразків справжнього класу *verified*, помилково віднесених системою до *unverified*, дозволив виявити три основні причини помилок, кожна з яких має свою природу та шляхи усунення.

5.5.1 Аналіз пропусків *verified* → *unverified*

Перша причина – стислі узагальнені повідомлення (≈ 5 з 16 випадків). Найбільш фундаментальна причина помилок – повідомлення з Telegram, що складаються з 1–3 слів і не містять достатньо інформативних термінів для статистичного аналізу. Типові приклади: «вибухи у Кривому Розі», «тривога у Києві», «приліт по Одесі». Для таких повідомлень навіть успішне співставлення з новинною статтею про відповідну подію дає низьку оцінку TF-IDF (0,15–0,25),

оскільки спільних значущих лем критично мало. Це є *фундаментальним обмеженням лексичних методів*: для коректної верифікації стислих повідомлень принципово недостатньо тексту самого повідомлення. Можливі шляхи розв’язання складаються з побудови контекстного розширення повідомлення з метаданих (час, локація, назва каналу) або застосування генеративних моделей для попереднього перефразування й розширення, проте обидва підходи виходять за межі поточного дослідження.

Друга причина – близькі промахи (≈ 8 з 16 випадків). Другою групою помилок є повідомлення, оцінка яких опинилася в проміжку 0,37–0,50 (тобто нижче порогу $\tau_{uncertain} = 0,50$). Це переважно повідомлення про події, що дійсно висвітлюються в новинах, але стилістично сформульовані інакше: інша лексика, перефразування, наявність цитат у одному з джерел і їх відсутність в іншому. Бонус за іменовані сутності витягує оцінки догори, але виявляється недостатнім для подолання порогу. Цей тип помилок принципово вирішуваний або зниженням порогу $\tau_{uncertain}$ (ціною зростання хибнопозитивних спрацювань), або застосуванням більш потужних семантичних моделей (потенційно – україномовного fine-tuned BERT).

Третя причина – принципово низькі оцінки (≈ 3 з 16 випадків). Найменшою групою є повідомлення, що отримали оцінки 0,25–0,27, тобто значно нижче порогу $\tau_{uncertain}$. Якісний аналіз показав, що це повідомлення зі специфічним лексичним складом (наприклад, ситуативний сленг військовослужбовців, цитування заяв з інших мов з частковим перекладом, повідомлення про вузькоспеціалізовані технічні події). Усунення таких помилок потребує специфічних стратегій. Наприклад, використання спеціалізованих доменних словників або моделей, що виходить за межі загального методу.

Таким чином, з 16 пропущених верифікованих повідомлень приблизно 50 % припадає на близькі промахи, які можна усунути технічними засобами, ще близько 30 % – на фундаментальні обмеження стислих повідомлень, і близько 20 % на специфічні крайні випадки.

5.5.2 Низька якість роботи на класі uncertain

Як показано в табл. 5.3 та на рис. 5.6, F1-міра на класі uncertain становить лише 0,267, що значно нижче за відповідні показники для двох інших класів. Поглиблений аналіз дозволяє визначити три фактори, що зумовлюють цей результат.

Обмеженість розмічених даних класу. Лише 28 зразків класу uncertain з 305 загалом. Це принципова статистична межа: будь-яка метрика, обчислена на 28 зразках, має широкі довірчі інтервали. Один-два неправильно класифікованих зразка змінюють метрику на 3–7 процентних пунктів. Розв’язанням є додаткова анотація, що дозволила б збалансувати клас, проте це виходить за межі поточної роботи.

Природна вузькість зони рішення. Клас uncertain знаходиться у вузькому інтервалі $final_score \in [0,50; 0,55)$ шириною лише 0,05 – це лише 5 % всього діапазону оцінок. Будь-який «шум» у конкретному обчисленні, від випадкової відсутності спільної рідкісної лемми до варіацій довжини документів, може зміщувати зразок на сусідній клас.

Архітектурний компроміс. Бонус за іменовані сутності, оптимізований для покращення Recall на класі verified, систематично «штовхає» зразки uncertain догори (15 з 28 зразків uncertain класифіковано як verified – див. рис. 5.6). Це є наслідком обраного багатокритеріального компромісу: оптимізація на macro-F1 дала загалом найкращий результат, але саме на класі uncertain ефект бонусу виявляється негативним.

Важливо ще раз наголосити, що низький F1 на класі uncertain не свідчить про дефект 3-класової схеми як такої. Уся скрипична діаграма на рис. 5.5 наочно демонструє, що клас uncertain має власний характерний розподіл оцінок, відмінний від двох інших, тобто він є реальним і відокремлюваним. Низька кількісна якість є наслідком обмеженості даних, а не помилковості концепції.

5.5.3 Загальні обмеження методу

Узагальнюючи виявлені у ході експерименту обмеження, перелічимо ключові:

- залежність від наявності першоджерела. Метод фундаментально не може верифікувати подію, про яку немає публікації в авторитетних ЗМІ у часовому вікні. Для оперативних повідомлень, що з'являються в Telegram раніше за публікацію в новинних ЗМІ, метод дає некоректний результат *unverified*, який потім має змінюватися під час повторного запуску або у фоновому процесі;
- залежність якості від мови. Усі удосконалення (морфологічна лематизація, виділення сутностей за орфографічним правилом, словник стоп-слів) спеціалізовані для української мови. Перенесення методу на іншу мову вимагатиме повторного налаштування цих компонентів;
- чутливість до обсягу повідомлення. Як показав аналіз помилок, метод істотно гірше працює на повідомленнях коротших за 5–7 слів. Це не є дефектом методу – це фундаментальне обмеження будь-якого підходу, заснованого на лексичній або семантичній подібності;
- відсутність вимірювання міжанотаторської узгодженості. Як зазначено вище, експеримент проведено на даних, розмічених одним анотатором. Це обмежує статистичну надійність результатів і робить висновки залежними від суб'єктивного тлумачення граничних випадків.

5.6 Висновки до розділу

Експериментальне дослідження підтвердило загальну ефективність розробленого методу автоматичної верифікації повідомлень з Telegram-каналів та дозволило зробити кілька важливих наукових висновків.

По-перше, серед трьох розроблених методів (TF-IDF, семантичний на основі MiniLM, ансамблевий) найкращим за відношенням «якість/обчислювальна складність» виявився TF-IDF верифікатор з удосконаленнями: $\text{Assurasy} = 81,0 \%$,

$F1_{\text{verified}} = 0,775$, $\text{macro-F1} = 0,647$. Семантичний метод показав значно нижчі показники, що пояснюється короткою довжиною повідомлень і відсутністю спеціалізованого тренування багатомовної моделі MiniLM на україномовних даних. Ансамблевий метод дав граничне поліпшення базових метрик (≈ 1 процентний пункт), що не виправдовує приріст обчислювальної складності.

По-друге, ключові вдосконалення TF-IDF верифікатора: корпусно-рівнева IDF, морфологічна лематизація `ru morphology3`, бонус за іменовані сутності та штраф за низьке покриття ключовими словами – у сукупності підвищили Recall класу `verified` з 59,6 % до 77,9 % при контрольованому зниженні Precision з 87,5 % до 77,1 %. Цей обмін методологічно виправданий для задачі побудови корпусу, де пропуск верифікованої новини є гіршою помилкою, ніж включення граничного зразка.

По-третє, 3-класова схема анотації (`verified/uncertain/unverified`) є самостійним методичним внеском роботи. Розподіл оцінок на рис. 5.5 емпірично підтверджує, що клас `uncertain` має власний характерний розподіл і не може бути зведений до граничного випадку бінарної класифікації. Низький F1 на цьому класі (0,267) пояснюється обмеженістю розмічених даних, а не дефектом схеми.

По-четверте, виявлено фундаментальне обмеження лексичного підходу для дуже коротких повідомлень типу «вибухи у Кривому Розі». Усунення цього класу помилок є логічним напрямом подальшого розвитку, що потребує комбінації з контекстними розширеннями або генеративними методами перефразування.

По-п'яте, загальна якість роботи системи ($\text{Accuracy} = 81,0\%$) є достатньою для практичного застосування в задачі автоматичного формування динамічних корпусів української мови. На прикладі обробки 1042 повідомлень з трьох українських Telegram-каналів продемонстрована придатність методу до використання в режимі, наближеному до реального часу.

Перспективи подальшого розвитку дослідження полягають у донавчанні нейромережевих моделей (україномовних BERT/RoBERTa) на зібраному корпусі, розширенні бази розмічених даних з вимірюванням міжанотаторської узгодженості та дослідженні гібридних схем, що поєднують лексичні методи зі стратегіями перефразування за допомогою великих мовних моделей.

ВИСНОВКИ

У ході виконання магістерської кваліфікаційної роботи було вирішено актуальну науково-практичну задачу розробки програмного забезпечення для автоматизованого створення та верифікації корпусів української мови на основі даних соціальних мереж. На відміну від існуючих рішень, які або обмежуються бінарною класифікацією, або потребують великих розмічених навчальних вибірок, у роботі запропоновано та реалізовано підхід, що поєднує два незалежні методи верифікації, ансамблеве комбінування та трикласову схему анотації, причому весь процес виконується без потреби в попередньо навчених класифікаторах.

За результатами проведеного дослідження можна зробити наступні висновки:

- проаналізовано стан предметної області та встановлено, що існуючі корпуси української мови (ГРАК, UberText) є переважно статичними та не забезпечують достатнього покриття оперативної лексики із соціальних мереж. Виявлено, що основною перешкодою для використання даних з месенджера Telegram є високий рівень інформаційного шуму та низька фактологічна достовірність контенту, а також гостра нестача розмічених наборів даних, типова для української як мови з обмеженими ресурсами;
- розроблено та реалізовано два незалежні методи автоматичної верифікації повідомлень: метод на основі TF-IDF з корпусно-рівневою IDF, бонусом за збіг іменованих сутностей і штрафом за низьке покриття ключовими словами; та метод на основі семантичних векторів багатомовної трансформерної моделі paraphrase-multilingual-MiniLM-L12-v2 з м'яким попереднім фільтром та бонусом за корроборацію декількома джерелами. Додатково реалізовано ансамблевий метод, що лінійно комбінує оцінки обох верифікаторів;
- запропоновано трикласову схему анотації (verified / uncertain / unverified), яка точніше відображає природну градацію відповідності між повідомленням з Telegram та новинами-першоджерелами, ніж бінарна

- класифікація. Клас *uncertain* не є оцінкою впевненості системи, а самостійною змістовою категорією, що описує часткове покриття подій;
- спроектовано модульну архітектуру програмної системи з чотирьох рівнів (збору, обробки, зберігання та аналізу даних). Використання принципу слабкої зв'язності компонентів забезпечує гнучкість системи та можливість її подальшого масштабування;
 - програмно реалізовано ключові компоненти системи мовою Python з використанням бібліотек *Telethon* (для асинхронної роботи з Telegram API), *feedparser* та *BeautifulSoup* (для збору новин з RSS-стрічок), *rumorphy3* (для морфологічної лематизації української мови), *scikit-learn* (для TF-IDF та обчислення косинусної подібності), *sentence-transformers* (для семантичних векторів). Зберігання даних реалізовано у форматі JSONL, що забезпечує природну відтворюваність експериментів без зовнішніх залежностей;
 - проведено експериментальне дослідження на власноруч розміченій вибірці з 305 повідомлень (104 *verified*, 28 *uncertain*, 173 *unverified*). За результатами *grid search* обрано оптимальні гіперпараметри. TF-IDF верифікатор досягає *accuracy* 81,0 % і F1-міри для класу *verified* 0,775; ансамблевий метод з оптимальними вагами (0,9 / 0,1 для TF-IDF та семантичного відповідно) показує співмірний результат при значно вищій обчислювальній вартості;
 - встановлено, що корпусно-рівнева IDF та морфологічна лематизація *rumorphy3* є ключовими факторами якості: вони підвищують *Recall* класу *verified* з 59,6 % (базовий метод) до 77,9 % (фінальний метод). Додавання бонусу за іменовані сутності з коефіцієнтом 0,14 виявилось оптимальним, тоді як бонус за корроборацію в TF-IDF емпірично шкодить (оптимальний коефіцієнт 0,0);
 - виявлено фундаментальне обмеження лексичного підходу: короткі або стилістично узагальнені повідомлення (наприклад, «вибухи у Кривому Розі») не можуть бути надійно верифіковані будь-яким методом на основі

лексичної подібності через брак інформативного контенту. Низька F1-міра класу *uncertain* (0,267) пояснюється не недоліком методу, а обмеженою кількістю розмічених прикладів цього класу (28 зразків) – це обмеження даних, що знімається додатковою анотацією.

Практичне значення отриманих результатів полягає у створенні інструментарію, що дозволяє формувати якісні динамічні корпуси «живої» української мови з автоматичною позначкою достовірності кожного зразка. Розробленою системою оброблено 1042 повідомлення з трьох українських Telegram-каналів за період березень-квітень 2026 р. та сформовано розмічений корпус, придатний для подальшого навчання та оцінювання україномовних NLP-моделей. Програмні рішення можуть бути використані для покращення моделей розуміння природної мови, моніторингу інформаційного простору та протидії дезінформації.

Перспективи подальшого розвитку роботи полягають у наступному:

- донавчання україномовної моделі типу BERT/RoBERTa на зібраному корпусі [23] для підвищення точності семантичного верифікатора;
- розширення набору джерел: збільшення кількості Telegram-каналів та новинних RSS-стрічок для отримання більш репрезентативного корпусу;
- автоматичне вимірювання міжанотаторської узгодженості (Cohen's κ) шляхом залучення додаткових анотаторів – це підвищить статистичну надійність експериментальної оцінки;
- розробка веб-інтерфейсу для візуалізації результатів верифікації та інтерактивного дослідження корпусу;
- інтеграція мовних моделей (LLM) для семантичного перефразування та формального узагальнення повідомлень перед порівнянням, що потенційно знизить вплив стилістичної варіативності.